

平成 30 年度修士論文

双腕ロボットによる
不透明な液体を定量的に注ぐに関する研究

大学院情報理工学研究科
情報学専攻

学 籍 番 号 : 1730069
氏 名 : DONG CHENYU
主任指導教員 : 末廣 尚士 教授
指 導 教 員 : 工藤 俊亮 准教授
提出年月日 : 平成 31 年 01 月 28 日 (月)

目次

第1章 緒言	10
1.1 研究背景	10
1.1.1 サービスロボット	10
1.1.2 料理ロボット	11
1.1.3 注ぐ作業	12
1.2 関連研究と考察	12
1.2.1 注ぐ作業	12
1.3 研究目的	12
1.4 研究概要	14
1.5 論文構成	14
第2章 提案手法/アルゴリズム	15
2.1 検討	15
2.1.1 体積の計算方法	15
方法1:力覚センサーによる方法	15
方法2:容器のモデルによる方法	16
まとめ	16
2.1.2 注ぐ手法	16
手法1:注ぐ容器を注目する	16
手法2:注ぎ先容器を注目する	17
注ぐ手法のまとめ	18

2.2	体積の計算	18
2.3	手法1：注ぐ容器を注目する	19
2.3.1	液面高さに基づく注いだ体積の計算	20
2.3.2	注ぐ動作のPD制御	20
	注ぐ時の傾き制御	20
	戻し動作	21
2.4	手法2：注ぎ先容器を注目する	22
2.4.1	注ぐ容器の傾き角度と注いだ量の関係による注いだ体積の計算	22
2.4.2	注ぐポイントの検出	24
	注ぐ口の検出	24
	注ぐポイントの検出	24
2.4.3	注ぐ動作のP制御	24
	注ぐ時の傾き制御	24
	戻し動作	25
第3章	システム概要	26
3.1	ハードウェア	26
3.1.1	双腕ロボット	26
	ツール座標系	27
3.1.2	力覚センサ	28
3.1.3	グリッパ	29
3.1.4	エンドエフェクタ	30
	その1	30
	その2	30
3.1.5	視覚センサ	31
3.2	ソフトウェア	32

3.2.1	librealsense	32
3.2.2	PCL(Point Cloud Library)	33
3.2.3	OpenRTM	34
	システム全体の接続構成	34
	RTC_PA10	35
	RTC_Gripper	35
	RTC_Realsense	36
	RTC_PointCloudProcess	36
	RTC 全体接続	37
第 4 章	提案手法の実装	40
4.1	点群モデルと取得	40
4.1.1	点群について	40
4.1.2	点群取得設備	40
4.1.3	容器点群モデルの取得する手法	40
4.1.4	多視点点群の合成	41
4.2	体積の計算	42
4.2.1	点群の分割	42
4.2.2	断面積の計算	42
4.3	手法 1 : 液面高さに基づく注いだ体積の計算	42
4.3.1	液面検出	42
4.3.2	変換テーブル	43
4.4	手法 1 : 注ぎ動作	43
4.5	注ぐ容器の傾き角度と注いだ量の関係による注いだ体積の計算	44
4.5.1	始まる液体体積の取得	44
4.5.2	注ぐ口の検出	44

断線端点の検出	44
断面の断線端点の検出	44
4.5.3 注ぐポイントの検出	46
4.5.4 注ぐ方向に対する出す量と傾き角度の関係の計算	47
ノイズ処理	47
傾き角度から出す量への変換テーブルの連続化	47
出す量から傾き角度への変換テーブルの計算	47
4.6 容器の掴み方	48
4.6.1 その一：注ぐ口のエッジを掴む	48
4.6.2 その二：外側から掴む	48
第5章 実験	49
5.1 実験条件	49
5.1.1 作業領域	49
5.1.2 容器	49
容器の識別	49
手法1	49
手法2	49
5.1.3 目標容器体積真値の測定について	50
5.1.4 ミルクティーの密度	50
5.1.5 秤について	51
5.2 共通実験	51
5.2.1 体積測定精度の実験	51
実験方法	52
実験結果	52
考察	53

5.3	手法1の実験	53
5.3.1	実験手順	53
5.3.2	注ぐ精度の評価実験1	54
	実験方法	54
	実験結果	54
	考察	54
5.3.3	注ぐ精度の評価実験2	54
	実験方法	55
	実験結果	55
	考察	56
5.3.4	考察	56
5.4	手法2の実験	57
5.4.1	角度と出る量の関係の予測精度の実験1	57
	実験方法	57
	実験結果	57
	考察	61
5.4.2	角度と出る量の関係の予測精度の実験2	62
	実験方法	63
	実験結果	63
	考察	63
5.4.3	注ぐ精度の評価実験3 TODO:実験3 未完成	64
	実験方法	64
	実験結果	65
	考察	65
5.5	考察	65

第 6 章	結言	66
6.1	まとめ	66
6.2	今後の展望	66

目 次

1.1 業種別の雇用人員判断 DI (「過剰」－「不足」)	10
1.2 世界のサービスロボット市場規模の推移及び予測	11
2.1 パラメーターの説明	18
2.2 PD 制御の説明	21
2.3 戻し動作の説明	22
2.4 注ぎ前の注ぐ容器	23
2.5 注いでいる時の注ぐ容器	23
3.1 ロボットシステム	27
3.2 ツール座標系	28
3.3 力覚センサ (オレンジの枠内)	29
3.4 力覚センサのデータシート (オレンジの枠内)	29
3.5 平行グリッパ (オレンジの枠内)	30
3.6 エンドエフェクタ 1	31
3.7 エンドエフェクタ 2	32
3.8 Realsense D435	33
3.9 全体の接続	34
3.10 RTC_PA10	35
3.11 RTC_Gripper	35
3.12 RTC_Realsense	37
3.13 処理の流れ	38

3.14	RTC.PointCloudProcess	39
3.15	RTC 全体接続	39
4.1	ConvexHull の結果	45
4.2	断線端点の計算	45
4.3	注ぐ口のノイズ (容器 3 の上から撮影)	46
4.4	線形補間	47
4.5	注ぐ作業が非常に困難	48
5.1	目標容器	50
5.2	目標容器であるお皿	51
5.3	使った秤	52
5.4	予測精度の実験 1 の結果 1	58
5.5	予測精度の実験 1 の結果 2	59
5.6	予測精度の実験 1 の結果 3	60
5.7	予測精度の実験 1 の誤差 1	61
5.8	予測精度の実験 1 の誤差 2	61
5.9	予測精度の実験 1 の誤差 3	62
5.10	表面張力	62
5.11	予測精度の実験 2 の結果	63
5.12	表面張力をなくした	64

表 目 次

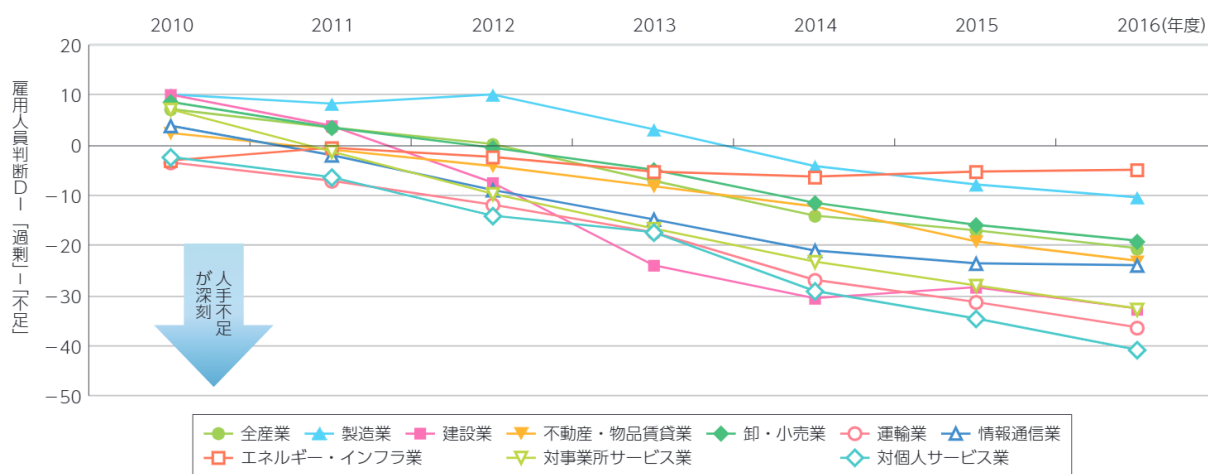
5.1	体積測定精度の結果	52
5.2	実験 1 の結果	55
5.3	実験 2 の結果	56

第1章 緒言

1.1 研究背景

1.1.1 サービスロボット

総務省平成30年版情報通信白書 [?] により、日本労働人口が今後減少すると予測している。その労働人口の減少により、今後多くの企業において人材不足が加速するであろう。特に大きく影響を受ける労働集約型の農林水産業や建設業、運輸・流通業やサービス業においては、既に経営課題として顕在化している。日本銀行が発表している全国企業短期経済観測調査（短観）の業種別計数によると、2014年以降は全ての業種において人手は「不足」となっており、特に建設業や運輸業、対事業者サービス業、対個人サービス業において深刻であることがわかる (図 1.1)。 [?]



(出典) 総務省「ICTによるイノベーションと新たなエコノミー形成に関する調査研究」(平成30年)
(日本銀行「企業短期経済観測調査」より作成)

図 1.1: 業種別の雇用人員判断 DI (「過剰」－「不足」)

また、総務省平成 30 年版情報通信白書 [?] により、その労働人口の減少の影響で、物流、ヘルスケア・介護、外食、店舗といった製造業以外のサービスロボットの市場規模は、負担の軽減等を目的として拡大が続いている。日本国内では物流での箱詰めやピッキング等でサービスロボットの活用が広がっているほか、店舗の接客やホテルの受付・クローク業務等一般消費者と直接的に接するサービスロボットの活用も増加している。 [?](図 1.2)。



図 1.2: 世界のサービスロボット市場規模の推移及び予測

更に、2020 年オリンピックは東京で開催するため、翻訳や案内のような対個人サービスの需要が拡大であろう。

1.1.2 料理ロボット

料理作業は人間にとって非常に重要な作業である。料理ロボットは人間を支援するサービスロボットの研究の一種として、世界中で研究が盛り上がっている。

1.1.3 注ぐ作業

料理作業において、注ぐ作業はよくある。例えば、液体調味料 (酢や醤油など) を入れることや料理に水を加えることなど。その注ぐ作業は単なる注ぎ先の容器に注ぐだけではなく、料理のため、定量的に注ぐ必要がある。定量ではないと、料理の味あるいは食感が変わってしまう。

1.2 関連研究と考察

1.2.1 注ぐ作業

ロボットによる注ぎ作業の研究はこれまでも行われており、Schenck ら [?] による RGB-D 画像と熱画像を用いたお湯を注ぐの作業、Kennedy ら [?] による数学的なモデルを用いた定量的に注ぐ作業、Mottaghi ら [?] による CNN を用いた画像中の容器の容積や液体の体積の推定、Gravot ら [?] らによる料理作業のプランニングとシミュレーションなどがあげられる。我々の研究グループも、計量スプーンを用いた調味料の計量作業 [?] を実現した。

液体を定量的に注ごうとしたとき、人間であれば計量カップや計量スプーンあるいは秤を使って決められた量の液体を量ってから容器に注ぐ。しかし、このような道具を使う方法では正確な分量を計測できる反面、作業の手順が増えてしまう問題がある。例えば、水をいくつかのコップに均等に分ける場合、容器ごとに水の体積を量る必要があるため、作業手順が増加する。ロボットによる作業の場合、計量カップなどを用いなくても、カメラ等を用いて容器の形状を計測することによって注がれた液体の体積を計算することが可能である。それによって、作業手順を減らすことが可能になる。

1.3 研究目的

以上の考察を踏まえ、本研究の目的は外部の道具が使わないまま、双腕ロボットによる液体を定量的に注ぐことである。その目的を実現するためにはまず以下の2つ課題を解決する

ことが必要である．

1. 注いた体積の測定．
2. 注ぐ動作の制御．

1について，定量的に注ぐため，注いた体積がわからないなら，「定量」とは言えない．したがって，注いた体積の測定はこの研究の中心であり，できるだけ容器の形や姿勢に限らない，汎用性が高い方法が求められる．

2について，注いた体積がわかっていても，注ぐ動作の制御が上手に出来ないなら，液体がこぼれることや一気に大きな量が入ることになるかもしれない．これは誤差の原因となってしまう．したがって，注ぐ作業を実行している時において，注ぐ容器の回転角度の制御や注ぐ動作の設計も重要である．

しかし，主な2つの課題が解決するだけでは定量的に注ぐことが実現できない．それ以外の課題もある．具体的には：

1. 容器の認識．
2. 容器モデルの取得．
3. 容器を掴む動作．

1について，容器の認識は更に2つ課題に分ける．その一は容器位置の検出．容器の位置が分からなければ他の課題（例えば，モデルの取得や容器を掴むことなど）は実現できない．その二はどの容器が注ぐ容器かとどの容器が注ぎ先容器かを定めること．どちらが注ぐ容器かがわからなければ，注ぎ作業が進めない．

2について，容器モデルの形式（関数で表すモデルや点群など）や容器モデルの取得する手段である．本論文が使用した方法は容器モデルが必要ため，容器モデルがないと完成できない．

3について，各種の容器に対して，適切な掴み方や掴みところを選択することである．注ぐ動作を制御するため，ロボットアームは必ず道具を使用し，容器を掴み，容器を計算した

姿勢に移動や回転させる．容器を掴めないと，計算した姿勢に移動できない，注ぐ動作の制御にならない．

上述の課題を実現することで，決められた量の液体を容器に注ぐことという目的を目指す．

1.4 研究概要

本研究は以下のことが実現した

1. テーブルにおいた注ぐ容器と注ぎ先の容器を識別し，注ぐ容器を掴み，注ぎ先の容器に指定した体積の液体を入れること．
2. テーブルにおいた注ぐ容器外側の形状と中に液体の体積を検出し，注いだ体積と注ぐ容器の傾き角度の関係を求め，注ぎ先の容器に指定した体積の液体を入れること．

1.5 論文構成

本論文の構成は次の通りである．第二章は注ぐ作業を実現するための検討と注ぐ作業の手法で提出．第三章は実験のためのロボットシステムについて説明する．第四章は二章で提案した手法の実装である．第五章は提案した手法を検証するための実験である．第六章は本研究のまとめと今後の課題を述べる．

第2章 提案手法/アルゴリズム

2.1 検討

本セクションは容器に液体を定量的に注ぐ方法について、検討をする。本研究最終的にはテーブルの上に注ぐ容器と注ぎ先容器があり、注ぐ容器から注ぎ先の容器に液体を定量的に注ぐ作業を実現することである。そして、研究目的に書いた通り、本研究では主な2つ課題だけは実現できない。それ以外の3つの課題を解決しなければならない。その部分は自分の調査により、既存手法があるため、このセクションで述べない。

2.1.1 体積の計算方法

研究目的1(注いだ体積を計測すること)を実現する方法は、体積を計算する方法により、様々な手段がある。(例えば、計量コップ、秤やビュレットなど)そして、セクション1.2注ぎ作業の考察により、料理ロボットにおける2つ体積を計算する方法が考えた。

方法1:力覚センサーによる方法

方法1はロボットアームの手先に力覚センサをつけ、注いた液体の重量が得られ、液体の密度で重量から体積に変換する。

この方法は、注ぐ容器の方の力覚センサを使うか、注ぎ先容器の方の力覚センサを使うか、2つ種類がある。

注ぐ容器の方は使いにくい。なぜなら、液体は流動性があるからである。その影響により、注ぎをしている間に、注ぐ容器の重心は液体の流動とともに変化している。そして、ど

んな持ち方でも、注ぐ時の回転や重心の移動の影響により、重力の軸における重量の計算が非常に困難になる。

注ぎ先容器の方は注ぐ容器の方より使いやすい。なぜなら、注ぐ容器の方と違い、注ぎ先容器が回転しない限り、注ぐ時に重心の移動は常に重力の軸の方向であるから。

方法 2: 容器のモデルによる方法

方法 2 は容器のモデルを仕様する方法である。容器のモデルを視覚センサーで計測し、スライスする。そして、各部分の体積を足し合わせることで、複雑な形状の体積が計算できる。

まとめ

力覚センサを使う方法が一番簡単な方法でと考えられる。しかし、本研究では、料理ロボットにおいて定量的に注ぐ方法であり、現在使用している力覚センサの精度は 12.76g (力覚センサ精度の計算については、セクション 3.1.2 に参照する) が足りないと思う。また、精度が足りるとしても、注ぐ時の液体の流動やロボットの振動により、測った結果がずれる可能性は高いと思う。

以上の考察を踏まえ、体積の計算は方法 2: 視覚センサーによる方法とする。

2.1.2 注ぐ手法

体積の計算方法は容器モデルによる方法を使用し、注いだ体積の測り方は、注ぐ容器の方を注目するか、注ぎ先容器の方を注目するか、2つの方法が考えた。

手法 1 : 注ぐ容器を注目する

手法 1 は注ぐ容器に入れた液体の液面高さを注目し、注ぐ時に液面高さを注いだ体積にリウタイムに変換し、注ぐ容器傾き角速度を制御量とする PD 制御。

料理によく使われる液体調味料 (酢や醤油や油など) は流動性が高い液体だと考えられる。流動性が高い液体において、大きいな外力がないなら、液面が常に水平面と平行する。その特性を利用し、重力の軸に沿い、注ぎ先容器モデルの底から液面まで間の体積が注いだ液体の体積である。そして、液面高さと容器のモデルが分かれば、底から液面の間の体積、つまり注いだ液体の体積が計算できるはず。

しかし、注ぎ作業を実現することには、液体の量を正確に計測することだけではできない。適切な量を注ぐためにロボットを制御することも必要である。

そこで、手法 1 に対して、制御する方法は目標体積と注いだ体積の差を誤差とし、注ぐ容器傾き角速度を制御量とする PD 制御である。なぜなら、手法 1 は注ぐ容器をモデル化が困難で、出す量と傾き角度の関係が計算できない場合において、PD 制御は確実に注いだ液体を目標体積に到達させる方法からである。

この手法の良いところは注ぐ容器の状態や形状がわからなくても、正確に注げることである。一方、注ぎ先の容器の形状がわからないなら、使えなくなる。

手法 2 : 注ぎ先容器を注目する

手法 1 の悪い点を解決するため、注ぎ先の容器に注目することがやめ、注ぐ容器に注目する。手法 2 は注ぐ容器モデルとそこにある液体を計測し、出た量と傾き角度の関係を計算することにより、入れた体積を計算できる。

そして、注ぐ容器において、注いでいる時、液面高さと注ぎ口の最低点の高さが同じことである。また、注ぎ作業において、注ぎ先容器が溢れない限り、注ぐ容器から出た量は必ず注いだ量である。この 2 つ制約により、注ぐ容器中の液体体積と注ぐ容器のモデルが分かれば、注ぎ出す量と傾き角度の関係が計算できる。その関係により、注ぐ容器を目標体積に対応する傾き角度に回せば、目標体積の液体が出る。

手法 1 と同じように、ロボットの制御が必要である。そこで、手法 2 は目標体積に対応する傾き角度が計算できるため、制御する方法はロボットアームを P 制御でその角度に傾く。

この手法は手法 1 の悪い点が解決できる一方、注ぐ容器の形状や中にある液体の量どちら

かがわからないなら，手法2も使えなくなる．

注ぐ手法のまとめ

手法1は注ぎ先の容器にある液体の液面高さを見，注ぐ容器の傾き角速度のPD制御することにより，注ぐ作業を完成する手法である．

手法2は注ぐ容器の形状と中にある液体の量を計測し，出た量と傾き角度の関係を計算し，目標体積に対応する終わる傾き角度に回転することにより，注ぐ作業を完成する手法である．

2つ手法を合わせ，注ぐ容器か注ぎ先容器かどちらかが計測できれば，注いた体積の測定が可能になる．そして，各手法に対応する注ぐ動作の制御と合わせて，最終的に液体を定量的に注ぐことを実現する．

2.2 体積の計算

本セクションは，液体を注ぐ手法の検討による体積の計算方法2(容器のモデルによる方法)の詳細について，説明する．

ここでは，任意な容器のある傾き角度 θ に対して，底からある高さ H まで間の体積(つまり図2.1 青い部分)を計算する．

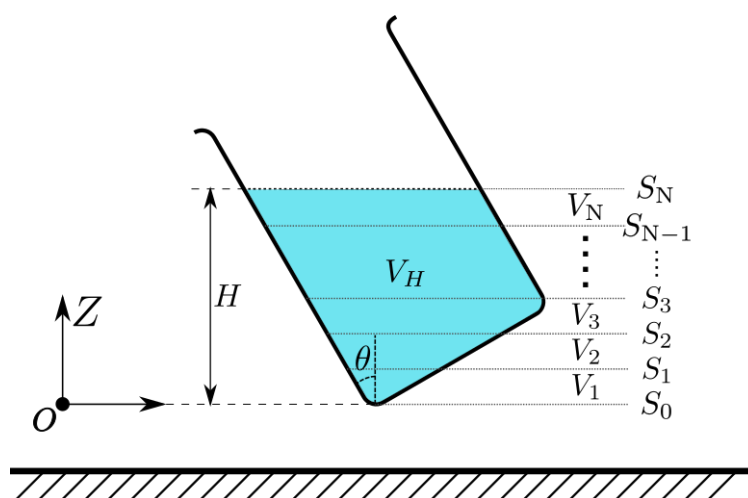


図 2.1: パラメーターの説明

あらかじめ容器のモデルが得られているとする．そして， H の部分の体積は下の式で決める：

$$V_{H,\theta} = \int_0^H S(h, \theta) dh$$

ここで， $S(h, \theta)$ はある傾き角度 θ において，容器のモデルを高さ h の水平面で切る断面積である．また， $S(h, \theta)$ も θ と h を変数とする容器のモデル関数である．

次に，上式の積分を計算するため，容器のモデルをいくつかの部分に分割し，各部分の体積を足し算することで計算する．容器の内部モデルがあり， z 軸に沿い，そのモデルをある高さ H の間で N 個の部分に分割する．各部分の高さを

$$h_{\text{diff}} = \frac{H}{N}$$

とおき， n 番目の部分の体積 V_n を以下の式で近似する．

$$V_n \approx \frac{(S_{n-1} + S_n) h_{\text{diff}}}{2} \quad (n = 1, 2, 3, \dots, N)$$

ここで S_{n-1} は n 番目の部分下断面の面積であり， S_n は n 番目の部分上断面の面積である．上の $S(h, \theta)$ の定義により，下の式を満たす：

$$S_n = S(h_{\text{diff}} \cdot n, \theta)$$

n 個の体積を全部足し算し合わせることで，上式の積分，つまり底からある高さ H まで間の体積 $V_{H,\theta}$ が以下のように計算される．

$$V_{H,\theta} = \int_0^H S(h, \theta) dh = \sum_{n=1}^N V_n \quad (2.1)$$

2.3 手法1：注ぐ容器を注目する

本セクションは手法1のアルゴリズムの詳細を述べる．

2.3.1 液面高さに基づく注いだ体積の計算

本セクションはセクション 2.1.2 で書いた液面高さに基づく注いだ体積の計算する方法の詳細について述べる。

注ぎ先容器の傾き角度 θ は注いでいる間で変わらないと仮定する。セクション 2.1.2 により、手法 1 はある傾き角度 θ に対して、底からある液面高さ $h_{\text{liquid},\theta}$ の間の体積 $V_{\text{liquid},\theta}$ を注いだ体積 $V_{\text{pour},\theta}$ とする。セクション 2.2 により、 V_{pour} は下の式を満たす：

$$V_{\text{pour},\theta} = V_{\text{liquid},\theta} = \int_0^{h_{\text{liquid}}} S(h, \theta) dh$$

この積分の計算はセクション 2.2 に参照する。

2.3.2 注ぐ動作の PD 制御

ここではセクション 2.2 のアルゴリズムにより、計算した注いた量に基づいて、目標容器に指定した量の液体を入れる動作を生成する手法について述べる。注ぎ動作は、考察に書いた「傾きの制御」のほか、さらに「戻し動作」を加え、二つの部分からなる。

注ぐ時の傾き制御

傾きの制御は、元の容器のモデルが未知の場合、システムのモデル化が困難であるため、初期状態 (図 2.2 の半透明の部分) から手先座標系を注ぐ容器の注ぎ口に設定し (図 2.2 の浅いオレンジの座標系)、その傾き角速度を PID 制御することで行う。しかし、I 制御が含まれていると、目標値に達しても積分項が注ぎ作業を続ける方向に働いてしまう。注ぎ作業では一度注いだ液体を元の容器に戻せないため、これは誤差の原因となってしまう。そのため、今回は PD 制御を用いる。

操作量である傾き角速度 ω は、現在の体積 V_{now} と目標体積 V_{tar} の差 V_{error} の比例、微分 (= 体積増加速度) D_{error} で以下のように決める。

$$\omega = k_p V_{\text{error}} + k_d D_{\text{error}}$$

誤差 V_{error} が大きい場合、上式の ω をそのまま使うと、一気に大量の液体が注がれてしまう可能性があるため、 ω の値に上限と下限を設定した。具体的には、 $-2^\circ/\text{s} \leq \omega \leq 2^\circ/\text{s}$ の範囲を超えた場合は、この範囲を超えない最も近い値に ω を設定する。注ぐ動作の終了条件は $V_{\text{error}} \leq 0$ で、到達したあとすぐに戻し動作を実行する。

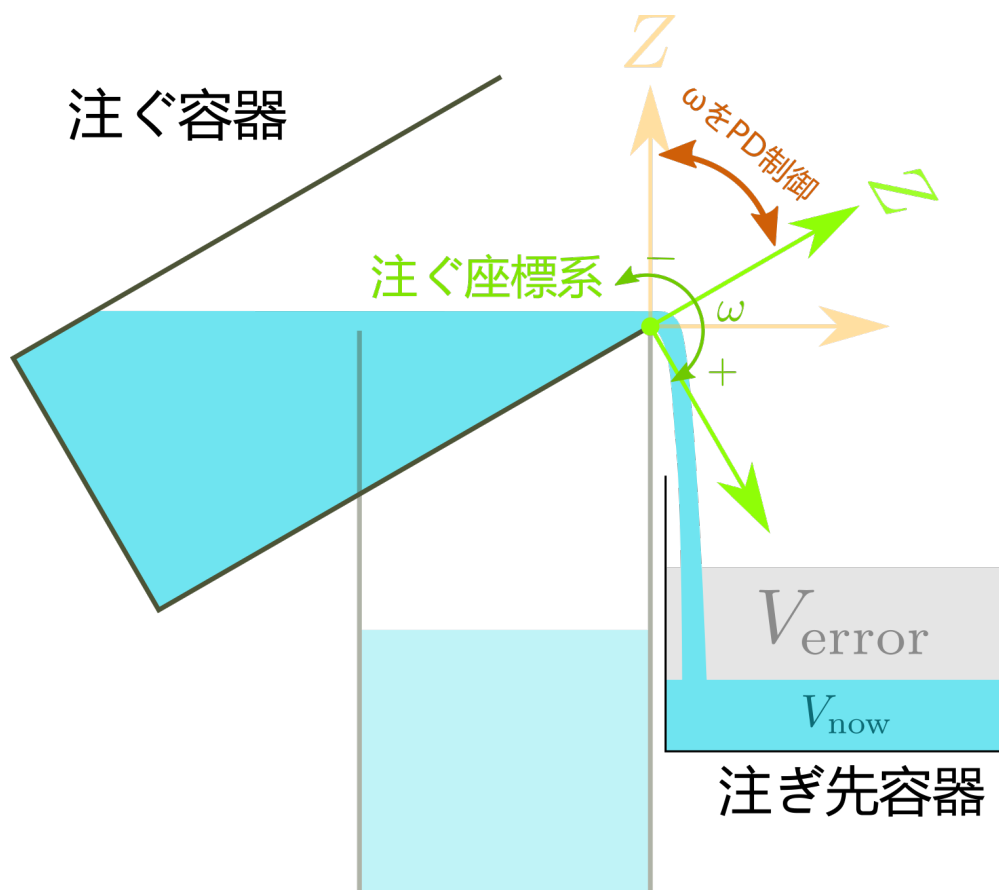


図 2.2: PD 制御の説明

戻し動作

戻し動作の目的は、目標体積に到達したあと液体の増加を出来る限り早く止めることである。具体的な動作は、PD コントローラを停止し (注ぐ容器は図 2.3 の半透明の状態)、手先座標系を戻し動作座標系 (図 2.3 のピンクの座標系) に設定し、 ω と逆の方向にテーブルに対して垂直になるまで (図 2.3 のオレンジの状態) 手先座標系を回転させる。

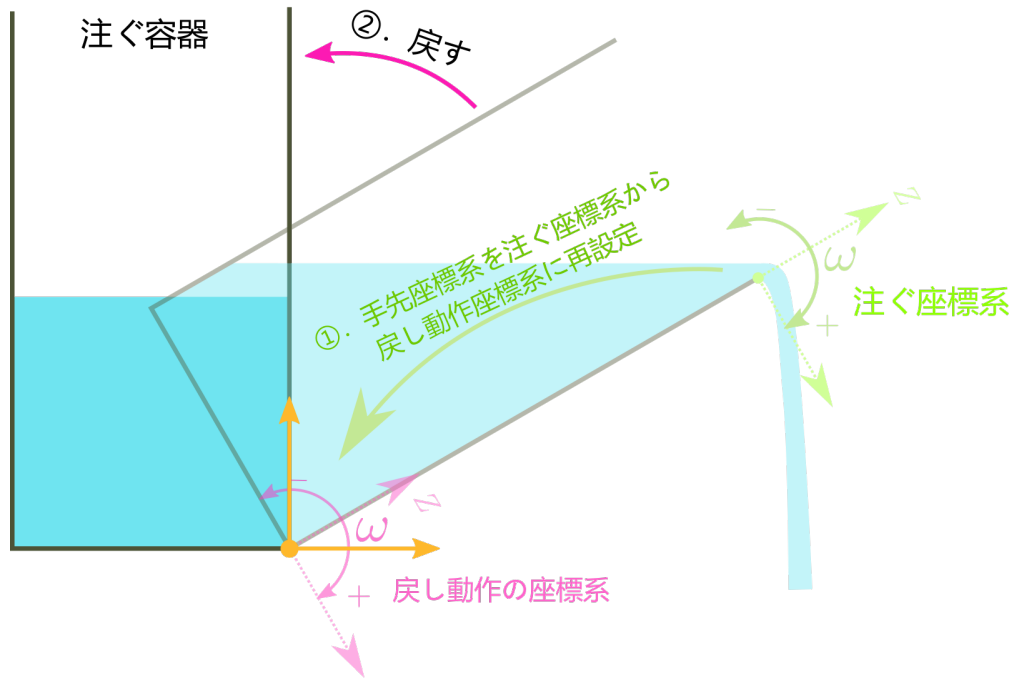


図 2.3: 戻し動作の説明

2.4 手法2：注ぎ先容器を注目する

2.4.1 角度と注いだ量の関係による注いだ体積の計算

本セクションは、手法2における注いだ体積の計算及び注いだ量と傾き角度の関係の計算について、説明する。

容器のモデルと中に入った液体の体積 V_{init} がすでに分かると仮定する。注ぎ前の時、今容器の傾き θ に対して、注ぐ口の高さ $h_{valid}(\theta)$ を注ぐポイント P_{pour} とモデルの一番低い点の z 軸の距離として定める。その高さは θ に関する関数である。図 2.4 のように、 $V_{valid}(\theta)$ は今の容器の傾き θ 角度においてこの容器の最大容積である (緑の枠)。 V_{init} は注ぐ作業が始まる前、予め中に入った液体の体積である (紫の枠)。そして、セクション 2.2 の方法により、 $V_{valid}(\theta)$ の計算は下の式で決める：

$$V_{valid}(\theta) = \int_0^{h_{valid}(\theta)} S(h, \theta) dh$$

この積分の計算はセクション 2.2 に参照する。 $h_{valid}(\theta)$ の計算は次のセクション 2.4.2 に参照する。

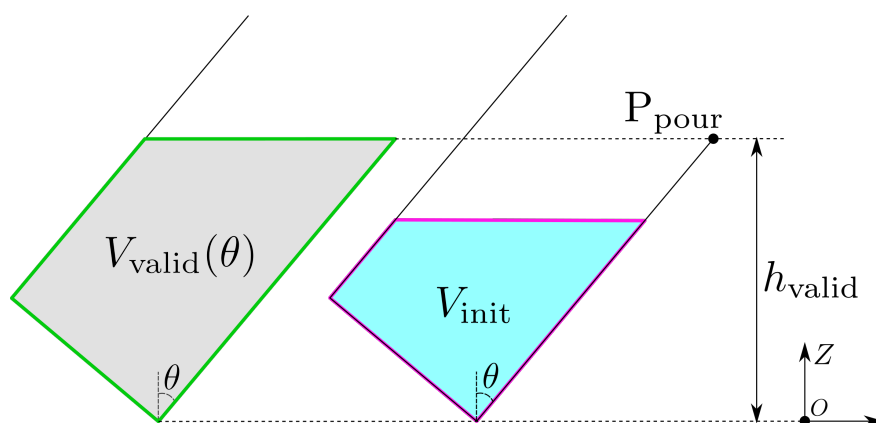


図 2.4: 注ぎ前の注ぐ容器

次は一回注いでいる時 (液体が出るときから), 下の図 2.5 のように :

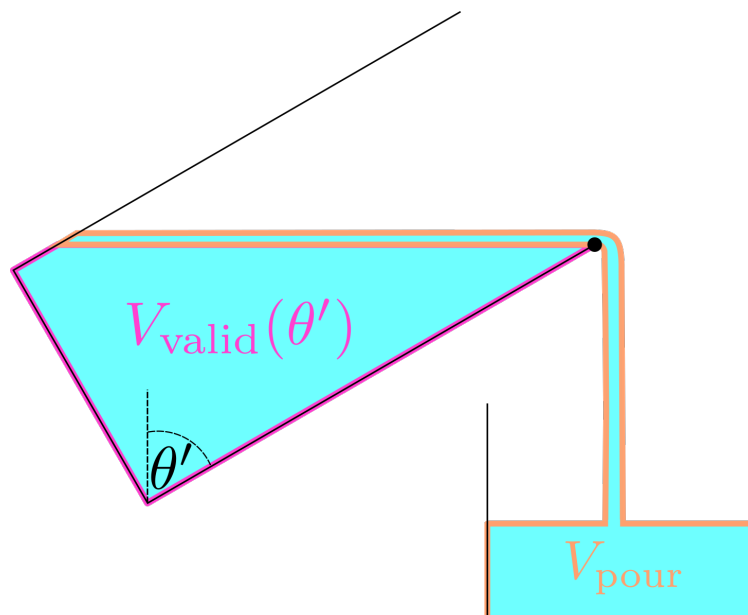


図 2.5: 注いでいる時の注ぐ容器

オレンジの枠の V_{pour} は注いだ液体の体積である. θ' は一回注いでいる時 (液体が出るときから) の注ぐ容器の傾き角度である. 注いでいる時には $V_{\text{valid}}(\theta')$ (紫の枠の部分) は常に残した液体の体積 V_{rest} と同じである. つまり :

$$V_{\text{valid}}(\theta') = V_{\text{rest}}$$

セクション 2.1.2 の 2 つ制約により, 下の関係が満たす :

$$\begin{aligned} V_{\text{init}} &= V_{\text{rest}} + V_{\text{pour}} \\ &= V_{\text{valid}}(\theta') + V_{\text{pour}} \end{aligned} \quad (2.2)$$

最後に, 出す量と傾き角度 θ の関係 $V_{\text{pour}}(\theta)$ は下の式で決める.

$$V_{\text{pour}}(\theta) = \begin{cases} 0 & \text{if } V_{\text{valid}}(\theta) > V_{\text{init}} \\ V_{\text{init}} - V_{\text{valid}}(\theta) & \text{Otherwise} \end{cases} \quad 0 \text{ deg} \leq \theta \leq 180 \text{ deg} \quad (2.3)$$

2.4.2 注ぐポイントの検出

セクション 2.4.1 により, 出す量と傾き角度の関係の計算を実現するためには, 注ぐポイント P_{pour} の検出が必要である. 注ぐポイントの定義は注ぐ口の最低点である. それを検出するためには, 本セクションはさらに「注ぐ口の検出」と「注ぐポイントの検出」2 つ部分からなる.

注ぐ口の検出

注ぐ口は容器の内部から外側に到達するエッジである. したがって, 容器をスライスすれば, 注ぐ口のスライスは必ず不連続な断線がある. それを利用し, 同じ容器の様々な姿勢において, 鉛直方向のスライスで, 不連続な断線の端点が探せ, 集めば, 注ぐ口が検出できる.

注ぐポイントの検出

注ぐ口が抽出でき, ある注ぐ方向に対して, 注ぐ口の最低点を探せば, 注ぐポイントが検出できる.

2.4.3 注ぐ動作の P 制御

注ぐ時の傾き制御

手法 2 の制御は容器注ぎ方向の傾き角速度を制御量とする P 制御である.

注ぐ容器と注ぎ先の容器をぶつからないように、手法1(セクション2.3)と同じ、ロボットの手先座標系を注ぐポイントに設定する。セクション2.4.1の方法で、目標角度 θ_{tar} はすでに計算されたため、注ぐ動作の制御はただ容器を注ぎ方向の傾き角度に傾けば完成である。

具体的には：現在の傾き角度 θ と目標値である目標角度 θ_{tar} の差をP制御の誤差 θ_{error} とする。そして、制御量である傾き角速度 ω は下の式を満たす：

$$\omega = k_p \theta_{error}$$

誤差 θ_{error} が大きい場合を防ぐため、手法1と同じく ω の値に上限と下限を設定した。

注ぐ動作の終了条件は誤差 $\theta_{error} \leq 0$ である。終了条件が到達したあと、液面高さと注ぐポイントの高さが同じまで、 t 秒を待つ。待ち終わったら、最後の戻し動作を実行する。

戻し動作

戻し動作は手法1(セクション2.3)の戻し動作と同じため、セクション2.3.2に参照する。

第3章 システム概要

この章は2章で提案した手法を実現するため構築した双腕ロボットシステムを紹介する。基本的に長野らが構築したシステムを用いた。

3.1 ハードウェア

3.1.1 双腕ロボット

三菱重工製7自由度の汎用ロボット PA10-7C を2台用いて、図3.1のような双腕ロボットシステムを構築した。

ロボットは、卓上での作業に従事し易いよう床面から約700[mm]の高さに設置した。各アームのベース座標系の原点は各アームの底の中央であり、各軸の向きはロボットから見た前方に x 軸, ロボットから見て左側に y 軸, 鉛直上方に z 軸となるように設定した。そして、今回の実験は両手同時に使う必要があるため、世界座標系が必要である。世界座標系は左ロボット(図3.1に座標系がついているロボット)のベース座標系の原点として、各軸の向きは左ロボットのベース座標系と同じように設定した。次に、回転について、本論文は xyz の經典順序のオイラー角度を使い、それぞれの回転角度は α, β, γ として表す。プラス回転方向は図3.1のように、回転軸を自分を指向し、逆時計回りの方向に設定した。また、各ロボットアームがお互いの作業を妨害しないことと同時に共同作業でもできるため、両アームが Y 軸で804[mm]の間隔を設置した。最後に、両アームの手先には、(図3.3、3.8、3.5)のように各種アタッチメントを装備した。

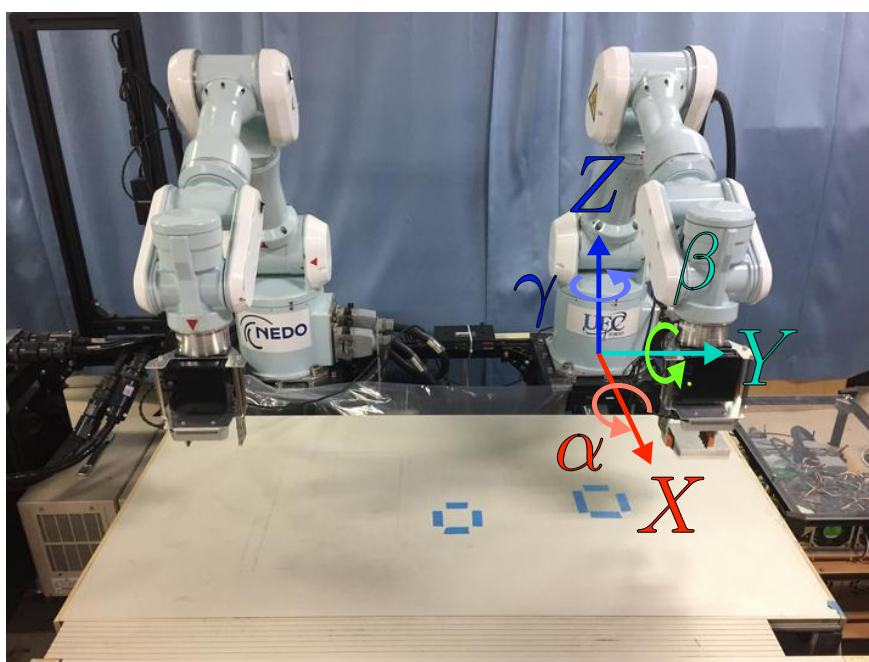


図 3.1: ロボットシステム

ツール座標系

ツール座標系はロボットが操る道具の座標系である。ロボットが道具を目的が達成できる位置や姿はるかにするため、ロボットの手先座標系を制御するより、操る道具の座標系を制御するのが遥かにしやすい。

PA ライブラリーはすでに PA10 の手先の姿勢を制御するプログラムが実現した。しかし、今回はツールを装着したままで実験が行い、直接にツールの先端の姿勢を制御するため、ツール座標系が設定した。

具体的には、PA10 手先の姿勢の位置は PA10 手先円盤の中心であり、回転はベース座標系から、Y 軸で 180 度を回った姿勢である。そして、ツール座標系は PA10 手先座標系の位置はツールの中心であり、回転は PA10 手先座標系から、Y 軸で 180 度を回った姿勢である。つまり、ツール座標系の回転はベース座標系と同じである。更に、各種のツールに対応するため、ツール座標系の姿勢はプログラムで調整できる。

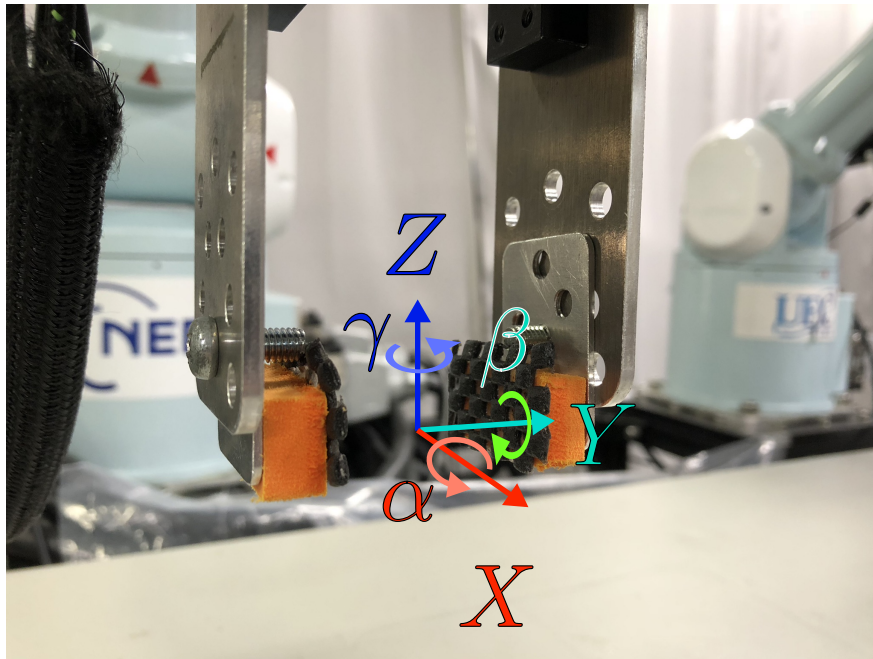


図 3.2: ツール座標系

3.1.2 力覚センサ

両腕の手首の部分に，Leprino 製の 6 軸力覚センサ PFS080YA501U6S(図 3.3) を装備した．

この力覚センサの性能パラメーターは (図 3.4)

このデータシートにより，我々のグループに使った力覚センサーの分解能力は

$$\begin{aligned}
 Resolution &= \text{定額荷重} \times \text{分解能} \\
 &= 500 + (1/4000) \\
 &= 0.125\text{N} \\
 &= 0.125\text{N} \div 9.8\text{N/Kg} \\
 &\approx 0.012755\text{Kg} \\
 &= 12.76\text{g}
 \end{aligned}
 \tag{3.1}$$

12.76g である．



図 3.3: 力覚センサ (オレンジの枠内)

PFS Series		030YA	055YA	055YA	080YA	080YS	100YA	100YS
定格荷重 (±)	Fx/Fy (N)	300	250	500	500	500	1500	1500
	Fz (N)	300	250	500	500	1000	3000	3000
	Mx/My (Nm)	4.0	6.0	10	20	40	100	150
	Mz (Nm)	4.0	6.0	8	20	30	80	120
許容過負荷 (±%R.C.)		200	200	200	200	200	200	200
非直線性 (±%R.O.)		1.5	1.5	1.0	1.0	1.0	1.0	1.0
他軸干渉 (±%R.O.)		3.0	3.0	2.0	2.0	2.0	2.0	2.0
分解能 (±R.O.)		1/2000	1/2000	1/4000	1/4000	1/4000	1/4000	1/4000
貸出機			○	○	○	○	○	○
価格(税別)		¥184,000	¥98,000	¥160,000	¥176,000	¥224,000	¥304,000	¥360,000

図 3.4: 力覚センサのデータシート (オレンジの枠内)

3.1.3 グリッパ

力覚センサの下についたのは平行グリッパである。平行グリッパは1E-HM01 HANDを使い、指先のアルミ板は長野さんのまま、ツールを保持するため、様々な姿勢で取り付けができるように穴を多数空けた。[?]



図 3.5: 平行グリッパ (オレンジの枠内)

3.1.4 エンドエフェクタ

今回の注ぐ作業において、各種の容器を持ち上げることが必要と想定され、指先のアルミ板に2つ種類のエンドエフェクタをつけた。

その1

図 3.6 のように、このエンドエフェクタの目的は：グリーパの開く距離が10[cm] しかないため、ボールやコップなどを掴むのは難しいと想定され、その時、容器のエッジを掴むようにする。そして、摩擦力を増加するためアルミ板の上に小さなスポンジをつけた。

その2

図 3.7 のように、このエンドエフェクタの目的は：10[cm] 以内の容器を確実に掴むことで、摩擦力を増加するためアルミ板の上に薄いスポンジをつけた。

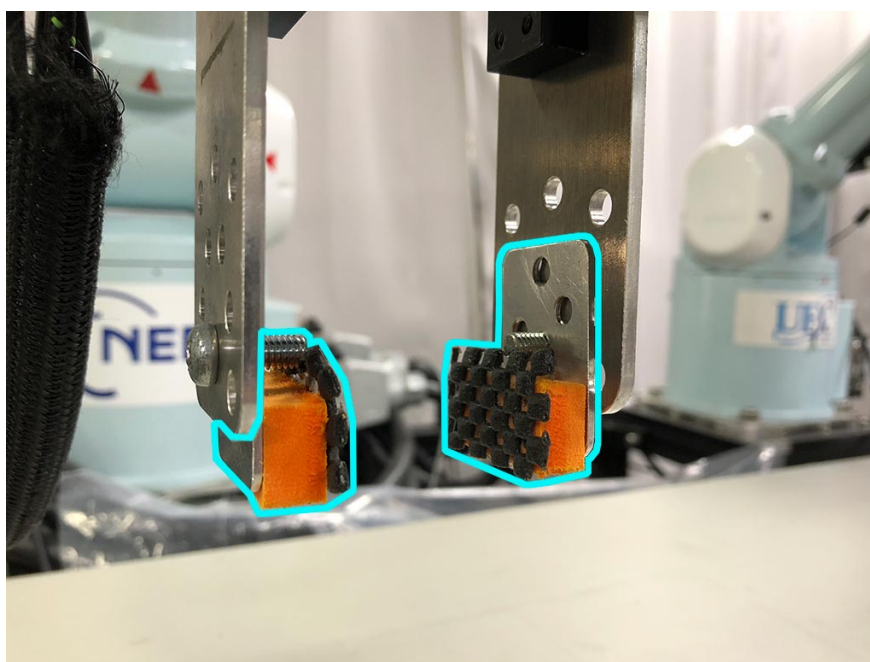


図 3.6: エンドエフェクタ 1

3.1.5 視覚センサ

図 3.8 のように，グリーパの前につけたのは本論文に使われる視覚センサ，Intel 製 RealsenseD435 (RGB-D カメラ) である．以前の視覚センサは RealsenseF200 (RGB-D カメラ) [?] であり，反射が強い材質 (陶磁器や液面やトマトの表面など) に対して，非常に大きいノイズが出る．そして，今年発売した RealsenseD435 が上述の問題が解決し，更に，精度も F200 より高い．したがって，RealsenseD435 が使えば，より完全や高精度の点群データが得られる．今まで，本研究グループのカメラは左手だけについている．今回は同時に違い角度から点群を撮るため，RealsenseD435 を両手に取り付けた．

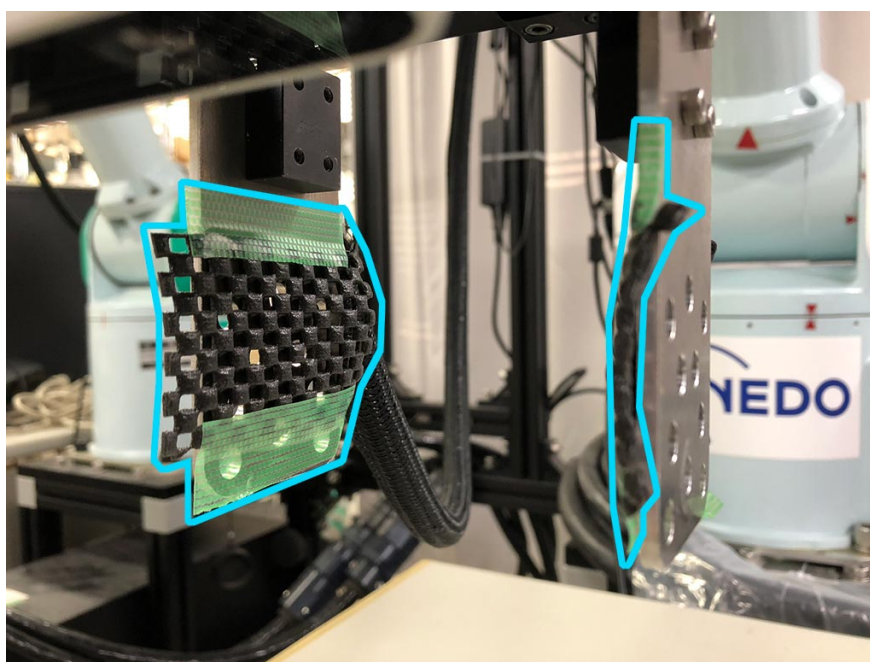


図 3.7: エンドエフェクタ 2

3.2 ソフトウェア

3.2.1 librealsense

librealsense は Intel 社が realsense シリーズのカメラを操作するためのクロスプラットフォーム SDK である。このライブラリはオープンソースで、GitHUB で日々開発している。使えるプログラミング言語は、librealsense は C++ で書いたが、C++ のほか、python や matlab や LabView などでも使える。そして、librealsense に提供したラッパーを使えば、他のライブラリーやゲームエンジン (例えば、PCL や OpenCV や Unity など) にセンサーのデータを簡単に送れる。

次に、機能面では、このライブラリはカメラにある各種センサーのデータの取得ことができる。それ以外、センサーの生データを処理する (例えば、デプス画像とカラー画像の合成やデプス画像から点群データに変換するなど) もできる。また、今 librealsense は主に 2 つバージョンがある：1.X と最新版の 2.X (本論文が各書く時点で最新版は 2.17.) である。1.X のバージョンは古いカメラ (F200, R200, LR200 と ZR300) のためのライブラリである。最

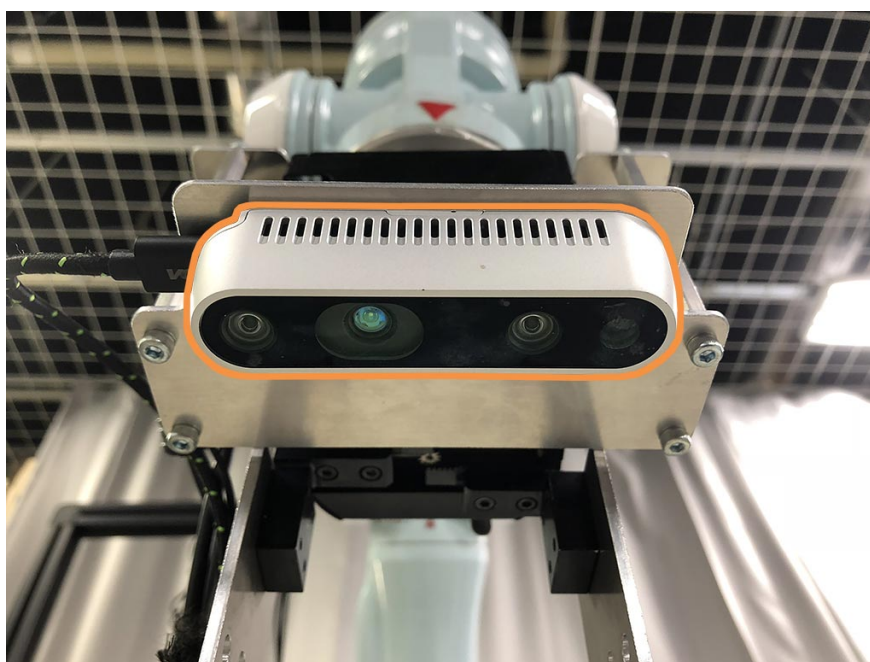


図 3.8: Realsense D435

新版の 2.X は新しいシリーズのカメラ (D400 と SR300) だけ対応する.

今回は RealsenseD435 が使うため, librealsense は 2.11 が使う.

3.2.2 PCL(Point Cloud Library)

PCL(Point Cloud Library) は大規模, オープンソースの 2D3D データを処理するライブラリーである. オープンソースのプロジェクトであるため, 日々更新していて, PCL ライブラリーは最新の論文にあるデータ処理手法を書き込むことが可能になる.

そして, 機能面については, PCL ライブラリーは点群データを処理することに必要な手法を全部モジュール化した. 例えば, よく使われているモジュールは, ノイズ処理のフィルターモジュール, 点群データの特徴を抽出するフィーチャーモジュール, 点群を合成させる registration モジュール, ある点の近傍をサーチする KD-Tree モジュール, 点群を分割するセグメンテーションモジュール, 点群を表示しやすいための可視化モジュールなど.

最新版は 1.9.1 が, 本論文は 1.8.0 を使う.

3.2.3 OpenRTM

本研究では RT-Middleware の実装のひとつである OpenRTM-aist [?] を用いてコンポーネント指向のロボット操作システムを構築した。以下では, RTC 毎にその詳細と全体的な接続について述べる。

システム全体の接続構成

図 3.9 に示した通り, 前節で述べた各ハードウェアを RT-Component(RTC) として作成し, RTC-handle [?] を用いてこれらをつなぎ合わせ, 全体を統括した。RTC-handle を実行している python のスクリプトで, ここから関数をコールすることで rtc-handle を介して各 RTC に指令を送る。

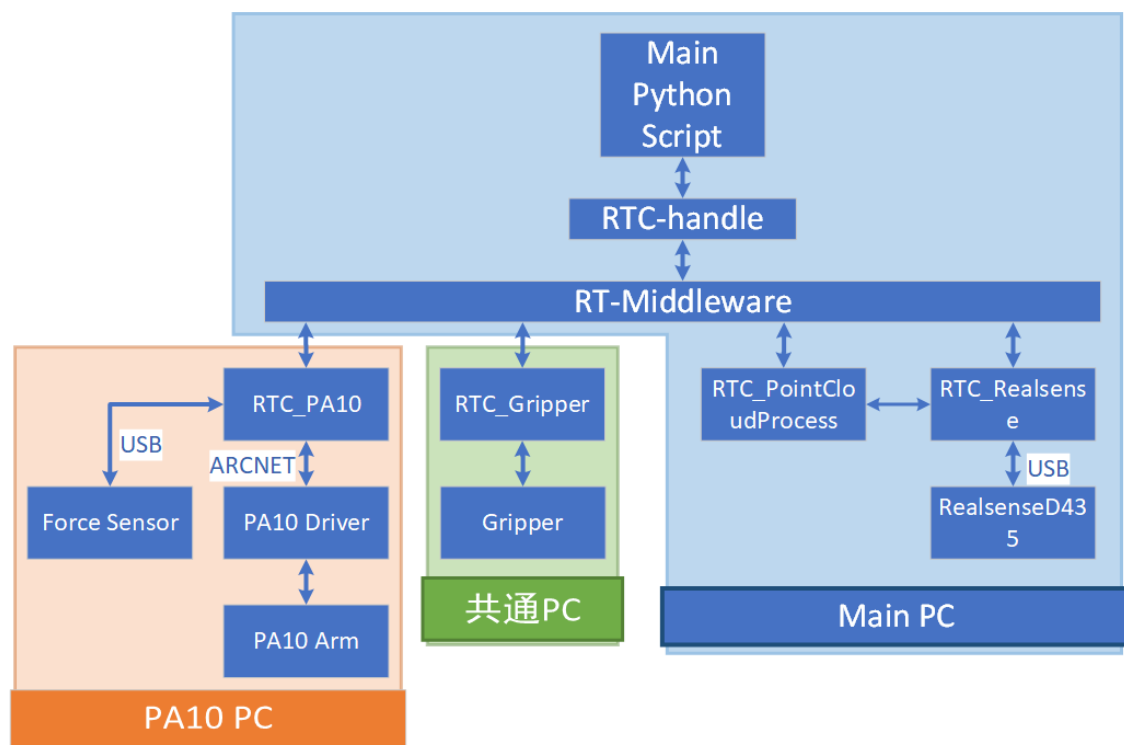


図 3.9: 全体の接続

RTC_PA10

RTC_PA10 は PA10 を操作するための RTC である。この RTC は独立の PC(図 3.9 の PA10 PC) でずっと実行している。この RTC は三菱が PA10 を操作しやすいため PA ライブラリを使っていて、ARCNET を通じ、PA10 のドライバと通信し、PA10 を制御する。そして、力のフィードバックで、ロボットの動きを制御するため、力覚センサーの操作もこの RTC に書き込んだ。力覚センサーは USB で PA10 PC に接続されている。

最後に、RTC_PA10 のインポートとアウトポートを紹介する。

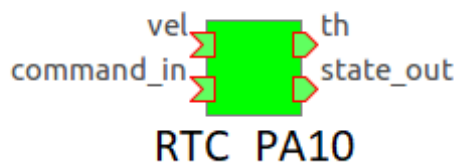


図 3.10: RTC_PA10

RTC_Gripper

RTC_Gripper は手先のグリーパを制御するための RTC である。具体的には、開くや閉じるや速度などの動きが制御できる。また、両手同時にとりつけたが、メンテナンスをしやすいため、各手のグリーパの RTC は各 PA10 PC で実行しなく、同時に共通 PC に実行している。

最後に、RTC_PointCloudProcess のインポートとアウトポートを紹介する。

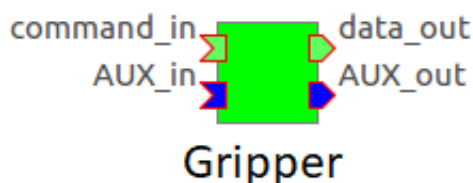


図 3.11: RTC_Gripper

RTC_Realsense

RTC_Realsense は前節で紹介した librealsense を使い、RealsenseD435 のデータを取得し、RTC_PointCloudProcess に送るための RTC である。この RTC はカラー画像、赤外線画像、デプス画像を RealsenseD435 から読み込んで、データタイプを RTC のタイプに変換し、RTC_PointCloudProcess に送る。また、前節で紹介した通り、librealsense のデータ処理機能も RTC に書き込んだ。それは：

1. デプス画像から点群データへの変換プログラム。
2. カラー赤外線画像から点群データにマッピングするプログラム。
3. デプス画像のノイズ処理プログラム。

次に、上に書いたデータ処理機能と RealsenseD435 からのデータの読み込みの速度を合わせるため、各データチャンネル(カラーや赤外線2つやデプス)をキューを設置した。これらのキューはいつも最新のデータをキューの末端に入れ続けている。もし、キューが満たれたら、新しいデータが来る時、新しいデータを末端に入れ、最も古いデータである先端のデータは捨てられる。そうすれば、キューから読み出したデータがいつも最新のデータである。ここで、データの遅延が少なくなるため、キューの大きさは5に設定する。

最後に、RTC_Realsense のインポートとアウトポートを紹介する。

RTC_PointCloudProcess

RTC_PointCloudProcess はセクション 3.2.2 で紹介した PCL を使い、RTC_Realsense が取得した点群データを処理する RTC である。

処理の基本的な流れは図 3.13 のように：

1. Python のメインスクリプトはサービスポートに通じ、処理命令及び命令に必要なパラメーターを RTC_PointCloudProcess に送る。
2. 命令により、RTC_PointCloudProcess のデータポートから点群を読み出す。

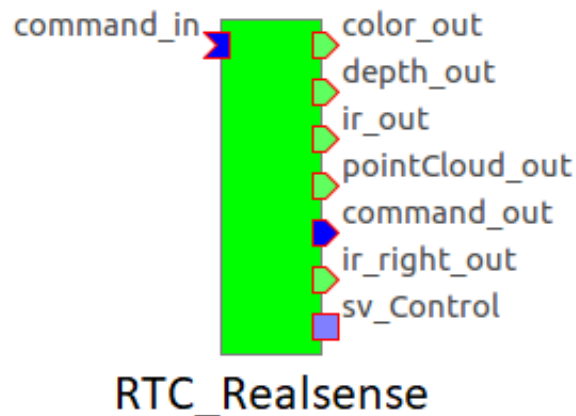


図 3.12: RTC_Realsense

3. アルゴリズムで点群を処理する.
4. 結果をサービスポートでメインスクリプトに返却する.

つまり、二章に紹介したアルゴリズム及び四章の実装に点群を処理することに関わる部分は全部この RTC に書き込んだ。

そして、今回は去年のロボットと違い、去年は左アームだけ Realsense がつけた。今年は同時に違い角度から点群を撮ることや両手の中のいずれかが取りにくくなる時を備えるため、両手に Realsense がつけた。それを応じて、RTC_PointCloudProcess も従来の一つの受信データポートから 4 つのポートに増加した。4 つのポートになる原因は両手以外のところ (天井カメラなど) にカメラを設置することを備えるためである。今回の実験は両手のカメラだけ、残した 2 つデータポートは接続しない。

最後に、RTC_PointCloudProcess のインポートとアウトポートを紹介する。

RTC 全体接続

RTC 全体接続状況は下の図で示した。

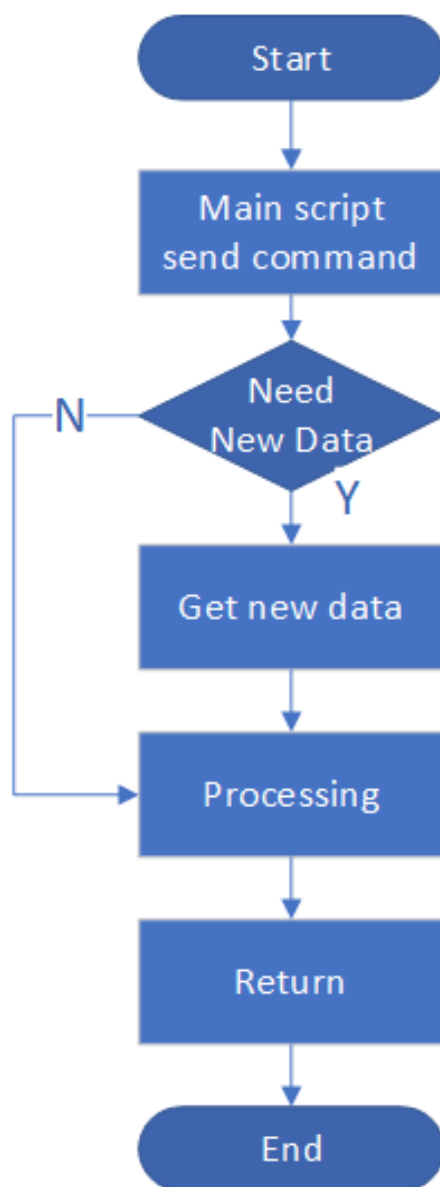
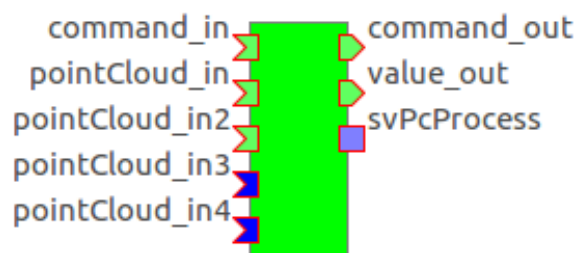


図 3.13: 処理の流れ



RTC_PointCloudProcess

図 3.14: RTC.PointCloudProcess

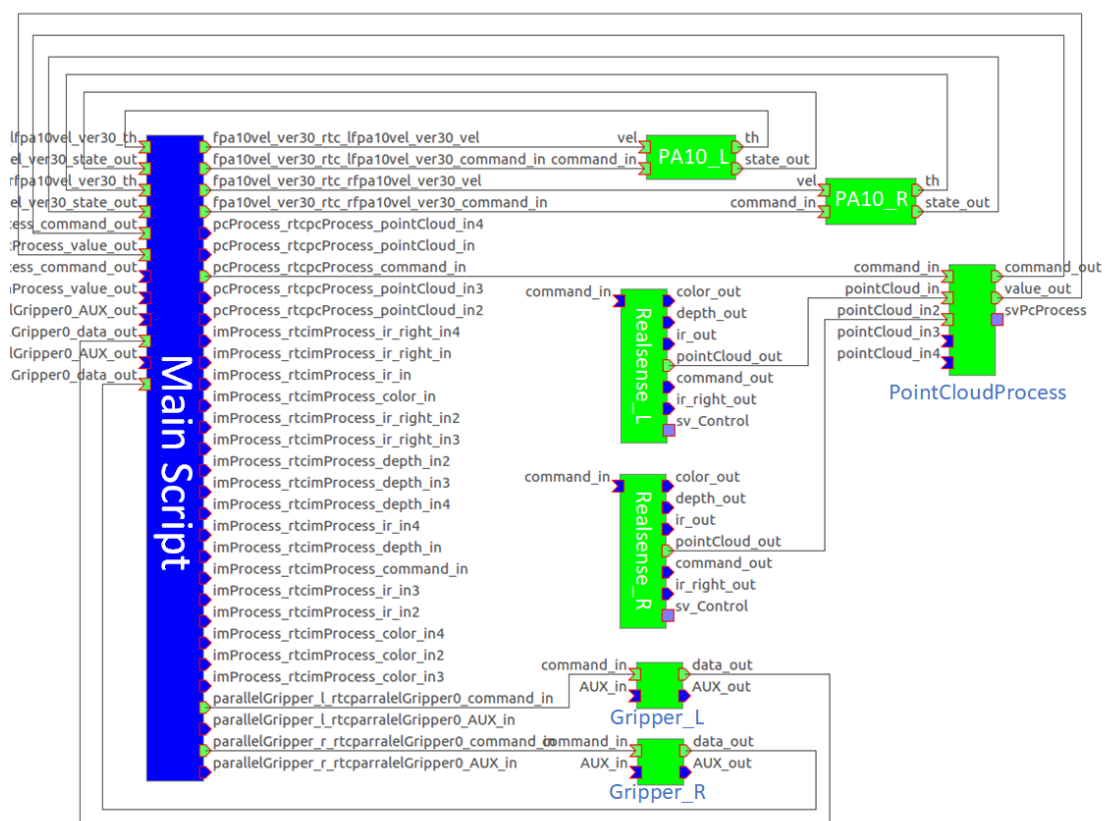


図 3.15: RTC 全体接続

第4章 提案手法の実装

4.1 点群モデルと取得

4.1.1 点群について

提案手法により，どんな手法でも，容器の体積を計算するため，容器の形状を計測することが必要である．つまり，容器の三次元の情報がないと，容器の形状がわからないである．そして，三次元の情報と言ったら，点群のことである．したがって，今回の実験は点群データが容器の形状データとする．

4.1.2 点群取得設備

次に，点群データを取得設備について，説明する．第三章で書いた通り，今回の実験では，ロボットアームに取り付けた Realsense で点群データを取得する．そして，点群データは RealsenseD435 というカメラのデプス画像と内部パラメーターで計算した．

4.1.3 容器点群モデルの取得する手法

容器の点群データから容器の体積を計算することは容器の完全な点群が必要である．そして，普通の容器にとって，完全な点群が取得するには，単視点の撮影はほぼ不可能である．そこで，今回の実験で完全な点群モデルを取得するため，多視点の撮影が行った．

具体的には，容器の壁が厚い容器にとって，外側の点群より，内部の点群のほうが重要である．したがって，視点の位置は内部の点群が取れる位置に設定しなければならない．

そこで、視点の位置の決め方は：RealsenseD435 は近いほど、精度が良くなるから、撮影安全 (被写体とぶつからないこと) を保証できる限り、撮影距離が近いほうがいい。上の条件と実験により、視点の位置 P は以下の条件を満たす円の上にある。

1. 円心は容器中心と容器の方向 (注ぐ口の法線ベクトル) で作った直線にある。
2. 円の法線ベクトルと容器の方向と同じである。
3. 半径 r は容器内部が見える数値に設定する。

$$\begin{aligned} P_x &= P_{\text{cup}_x} \\ P_y &= P_{\text{cup}_y} \\ P_z &= H + 20\text{cm} \end{aligned} \tag{4.1}$$

ここで、 H は容器の高さ、 L は容器短い辺の長さ。 θ は視点の位置 P のパラメーターで、円のどの位置を決める。

4.1.4 多視点点群の合成

点群の合成は多視点点群を一つになる過程であり、各不完全な容器の点群データを完全になることである。

合成する方法については、ICP (Iterative Closest Point) という近い点群の間のいち回転関係を求める方法がある。しかし、ICP は初期位置に敏感する方法で、不安定だと思うから、今回は使わない。

そこで、今回の研究はカメラがロボットの手先についているから、ロボットの手先の位置がわかれば、カメラの位置姿勢がわかる。そして、各視点の点群を全部ロボットの手先の位置で世界座標系に変換できれば、合成できる。したがって、上述の方法により、左手である視点で撮った点群を世界座標系に変換することは下の式を満たす：

$$P_w = {}^wT_{lb} * {}^{lb}T_{lh} * {}^{lh}T_c * {}^cP \tag{4.2}$$

ここで、 P_w は世界座標系に変換した点群である。 ${}^wT_{lb}$ は左手ベース座標系から世界座標系に変換する変換行列。 ${}^{lb}T_{lh}$ は左手の手先座標系から左手ベース座標系に変換する変換行列。 ${}^{lh}T_c$ はカメラ座標系から左手の手先座標系に変換する変換行列。 cP は RealsenseD435 で取得したカメラ座標系の下の子群である。また、第二章の 3.1.1 セクションの真ん中で書いた通り、世界座標系は左ロボットのベース座標系である。つまり、今回の実験で、 ${}^wT_{lb}$ は単位行列である。次に、右手が撮った点群について、第二章の 3.1.1 セクション最後の段落両手の関係を参照し、左手ベース座標系から右手ベース座標系への変換行列 ${}^{lb}T_{rb}$ を求め、下の式で合成する。

$$P_w = {}^wT_{lb} * {}^{lb}T_{rb} * {}^{rb}T_{rh} * {}^{rh}T_c * {}^cP \quad (4.3)$$

4.2 体積の計算

4.2.1 点群の分割

本セクションは今回の実験により点群の分割方法を説明する。今回の方法は各部分の高さを固定し、今回は

4.2.2 断面積の計算

分割した各部分の点群を分割平面に投影し、PCL にある ConvexHull というモジュールで、投影した点群 2D 上の面積を断面積とし、計算する。

4.3 手法 1：液面高さに基づく注いだ体積の計算

4.3.1 液面検出

本セクションはセクション 2.3.1 の方法が必要とする液面高さの取得について説明する。

最初は撮影した点群の中に液面検出に関係ない点を除去する。液面検出は注ぐ動作と一緒に動作しているため、検出は高いリアルタイム性が必要と考えられる。そこで、容器の世界座標系下の AABB バンディングボックスを使用し、PCL の PassThrough フィルターで、注ぎ先の容器の上から撮影した点群に対して、バンディングボックス以外の点を全部除去する。

最後は残した僅かな注ぎ先の容器の点群を PCL の平面抽出モジュールを使用し、最大点数の平面を液面とし、それを抜き出し、それらの点の z 軸の平均を液面高さとする。

4.3.2 変換テーブル

プログラムを加速させるため、液面高さから入れた体積への変換テーブルが作成した。セクション 2.2 で提案した手法は各断面の面積を計算する必要ため、その手法はリアルタイムで計算するのが難しいと考えられる。そこで、液面の高さからその高さに対応する体積への変換テーブルを容器のモデルを分析するの時に作成しておく。注ぐ際には、液面の高さを計測すれば、テーブルを参照することで注がれた液体の体積が求められる。

また、注ぐ際には、液面の高さはほぼ連続で変化していく。しかし、変換テーブルは連続ではない。そこで、連続な液面の高さに対して、最終の体積の結果は線形補間を使い、下の式で決める。

4.4 手法 1：注ぎ動作

実装は手法のままである。

4.5 角度と注いだ量の関係による注いだ体積の計算

4.5.1 始まる液体体積の取得

セクション 2.2 や 4.3.1 や 4.1 に書いた方法で、初期液体の体積が計算できる。

しかし、容器の初期状態はすでに液体が入る状態であるため、直接に容器の内部点群を取得ところが困難である。そこで、今回手法2の実験(セクション5.4.3)は外側の点群を容器の点群として、使用する。

4.5.2 注ぐ口の検出

注ぐ口の検出の流れは：

1. 注ぐ容器のモデルを選択された変換行列で変換する。
2. その姿勢において、点群モデルを z 軸に沿い、分割する。
3. 分割した各部分において、断面の断線端点を検出する。
4. 検出した点を??の逆変換行列で変換する。
5. 全部の姿勢が終わるまで、??から??まで繰り返す。

断面の断線端点の検出

断線端点は断面の形状の線に沿い、隣接する2つポイントの距離がしきい値より大きい点である。

上の定義が使用すれば、すぐ断線端点が検出できる。しかし、今回の点群の順がランダムで、直接に使えば、結果が違ふ。そこで、PCLのConvexHullというモジュールを使用し、容器のスライス点群エッジの点を計算し、それらの点の順を直す。図4.1で、赤いポイントはConvexHullの結果、青い線はConvexHullの結果によるスライス点群のエッジである。

次に、ConvexHullの結果から、隣接する2つポイントを取り出し、それらの間の点の分布を計算する。計算する方法は：

1. 隣接する2つポイントで確定した直線(図4.2の青い線)において、半径2.5[mm]の範囲内(図4.2の緑色の線)の点をすべて取り出す。

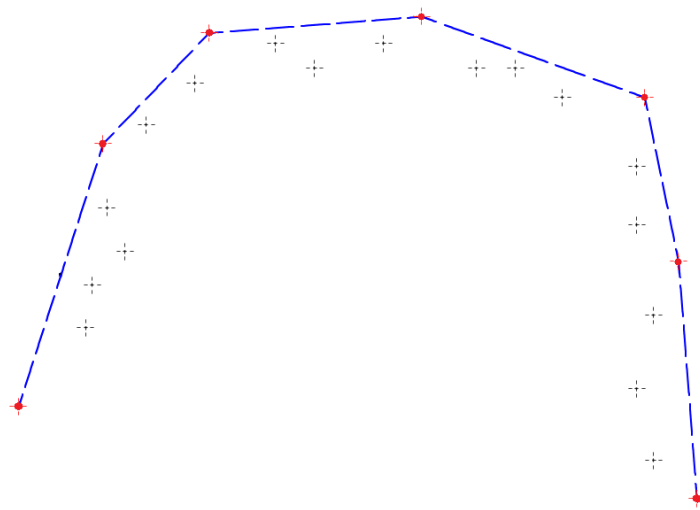


図 4.1: ConvexHull の結果

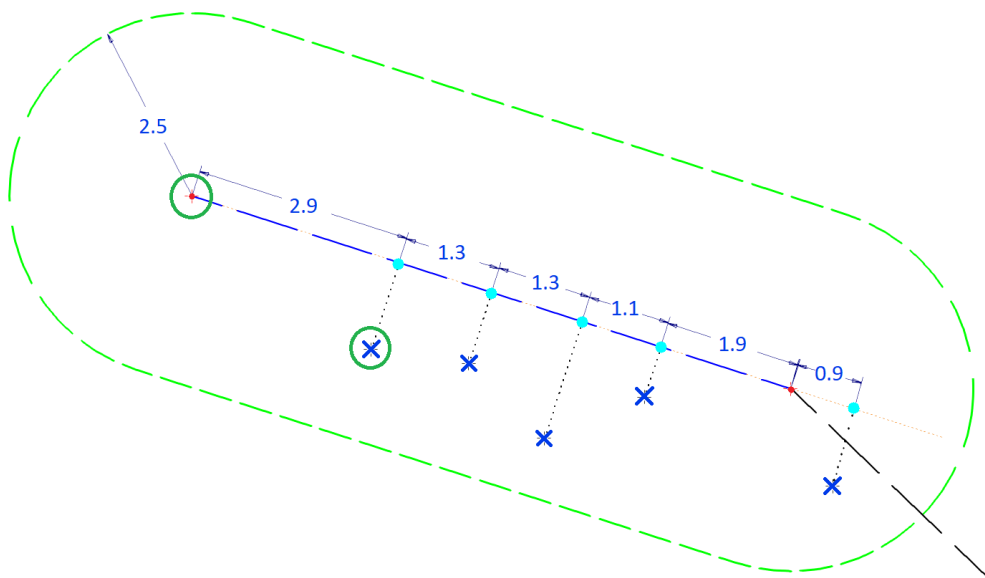


図 4.2: 断線端点の計算

2. 取り出す点をさっきの直線に投影する (図 4.2 の青い点).
3. 各投影した点の間の距離を計算する (図 4.2 の寸法).
4. 投影した点から隣接する 2 つポイントの距離を足し算した結果が隣接する 2 つポイントの間の距離より大きければ, この点は隣接する 2 つポイントの間の点ではないため, 計算手順から除く.
5. 投影した点の間の距離がしきい値より大きければ, それらの点を断線端点とする (しきい値は 2.5[mm] なら, 図 4.2 の深い緑の円が囲まれた点).

ノイズ処理

上の方法で計算した注ぐ口はノイズが必ずある (図 4.3).

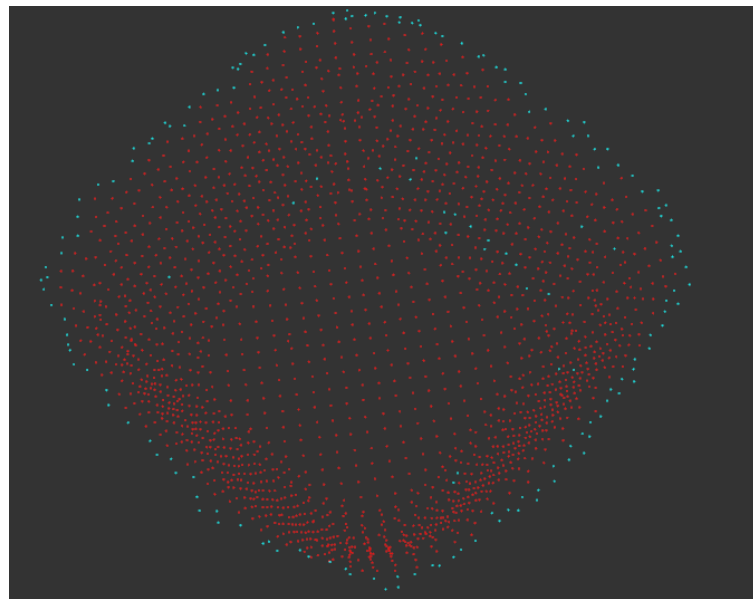


図 4.3: 注ぐ口のノイズ (容器 3 の上から撮影)

処理しないと, 次の注ぐポイントの検出に影響がある. そこで, PCL の EuclideanClusterExtraction というモジュールと使用し, 注ぐ口の点群をクラスタリングする. そして, クラスタの中で, 平均高さと点数が一番高いクラスタは注ぐ口とする.

4.5.3 注ぐポイントの検出

検出した注ぐ口を使用し、注ぐ方向に回転しながら、各傾き角度の注ぐ口の最低点を取り出す。そして、PCLのEuclideanClusterExtractionというモジュールを使用し、取り出した点をクラスタリングする。最後に、クラスタの中で、平均高さが一番低いクラスタの中心は注ぐポイントとする。

4.5.4 注ぐ方向に対する出す量と傾き角度の関係の計算

ノイズ処理

セクション2.2や2.4.2や4.5に書いた方法で、傾き角度から出す量への変換テーブルがすでに計算できる。

しかし、点群モデルのノイズの影響で、傾き角度の増加とともに、出す量が減少することもある。また、注ぎ作業において、傾き角度の増加に対して、出す量が減少することが不可能である。したがって、それを処理しないと、誤差の原因となってしまう。

そこで、傾き角度から出す量への変換テーブルにおいて、線形補間を使用し、減少が始まると終わるの間のデータを入れ替わる。

1	2	3	4	5	4	4	3	2	10	11	12	13	14	15
					↓	↓	↓	↓						
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

図 4.4: 線形補間

傾き角度から出す量への変換テーブルの連続化

変換テーブルは変換できるのは離散化したデータ。しかし、実際の時、傾き角度が連続な変数で、直接に使用できない。

そこで、線形補間を使用し、変換テーブルの間のデータを求める。

出す量から傾き角度への変換テーブルの計算

今まで、傾き角度から出す量への変換テーブルの計算と修正をした。しかし、実際に注ぐ時に、逆変換、つまり、出す量から傾き角度への変換テーブルが必要である。

そこで、出す量と傾き角度の関係が必ず増加という制約を利用し、バイナリサーチを使用し、出す量に対応する傾き角度を探す。

4.6 容器の掴み方

掴み方はセクション 3.1.4 二種類のエンドエフェクタに応じて、二種類がある。その一は大きいな容器に対して、注ぎ口のエッジを掴むこと。その二は小さいな容器に対して、外側から掴むこと。

4.6.1 その一：注ぐ口のエッジを掴む

セクション 2.4.2 で提案した方法で注ぎ口のエッジを検出した後、

4.6.2 その二：外側から掴む

PCL 主成分分析のモジュールを使い、掴む容器の中心を分析し、ロボットのツール座標系を容器の横から中心と合成し、グリーパが閉めれば、容器が掴まれた。

なぜ横から掴むのは、上から掴むなら、図 4.5 のように、ロボットの手先とテーブルや容器がぶつかる可能性があるため、注ぐ作業が非常に困難になる。また、注ぎ作用において、注ぎ先容器が掴まれ注ぐの場合より、注ぐ容器を掴んだ場合のほうが多い。



図 4.5: 注ぐ作業が非常に困難

第5章 実験

5.1 実験条件

5.1.1 作業領域

目標容器の内部モデルを得るためには，物体の 360 度の点群が必要となるので，注ぐ作業のできる範囲は 300 mm×400 mm 程度である．

5.1.2 容器

容器の識別

ロボットから見た右側の容器は注ぐ容器，左側は注ぎ先の容器である．

手法 1

手法 1 の実験では，内容量が 1000 ml の牛乳パックから液体を別の容器に注いでいく．注ぐ目標容器として図 5.1 に示す 3 つの容器を準備した．また，それぞれの体積は 290 ml, 120 ml, 88 ml である．

手法 2

手法 2 の実験では，注ぐ容器：容器 2 と容器 3，半分切った牛乳パック．注ぎ先の容器：お皿．



図 5.1: 目標容器

5.1.3 目標容器体積真値の測定について

水道水の密度は約 1.000g/cm^3 であり，したがって，目標容器の体積 (*in ml*) は数値的に目標容器に入った水道水の重量と等しい．測定方法はまず被測容器の中身をきれいにし，水の表面張力をなくすため，洗剤をエッジに薄く塗る．そして，被測容器を水平が調整した秤に乗せて，秤をゼロ表示にする．次に，水を入れてながら，液面を目視で確かめて，液面と被測容器のエッジと平行するまで，注ぎ続ける．最後は，液面と被測容器のエッジと平行する時秤の結果 m_{water} は $V_{\text{cup}} = \rho_{\text{water}} * m_{\text{water}}$ 式で被測容器の体積を計算する．また，一つの容器に対し，三回測定した結果の平均を容器の体積とする．

5.1.4 ミルクティーの密度

今回の実験では，注ぐ液体としてミルクティーを使用した．実験では，注がれた液体の体積を重量から求めるが，使用したミルクティーの密度は未知であったため，予め測定を行った．

ミルクティー密度の測定方法は，体積を量った容器にミルクティーを容器を満たすまで注ぎ，重量の増加分から密度を計算した．このとき，液体の表面張力による誤差をなくすため，洗剤を容器の縁に薄く塗った．以上の手順により求められたミルクティーの密度は 1.0276g/cm^3



図 5.2: 目標容器であるお皿

であった。

5.1.5 秤について

重さの測定に使った秤は TANITA 製 KD-320 である。精度は測定範囲によって異なり，0–300g: $d=0.1\text{g}$; 0–1500g: $d=0.5\text{g}$; 1500–3000g: $d=1.0\text{g}$ となっている。

今回の手法 1 では，容器 1 を使用した際は 0.5g の精度で計測し，容器 2,3 を使用した際には 0.1g の精度で計測した。

手法 2 では，容器 2,3 を使用するため，全部 0.1g の精度で計測した。

5.2 共通実験

5.2.1 体積測定精度の実験

この実験では，プログラムの注ぐ動作の部分分離し，予め決めた体積の液体を入れて，提案した手法で入った液体の体積を推定することにより，体積測定精度を評価する。



図 5.3: 使った秤

実験方法

容器 1 に 50 ml から 250 ml まで 50 ml の間隔で液体を注ぎ，プログラムによって求められた推定値を記録する．実験は各体積に対して 5 回行い，推定値の平均と標準偏差を計算する．

実験結果

体積測定実験の結果を表 5.1 に示す．

表 5.1: 体積測定精度の結果		
実際の体積	推定値平均	標準偏差
50	49.42	1.59
100	100.34	2.05
150	149.24	2.44
200	198.8	3.78
250	249.82	4.9

考察

推定された結果の平均値と標準偏差から、体積測定精度は実際の体積の2%ほどであることが分かる。

5.3 手法1の実験

5.3.1 実験手順

本セクションは手法1のプログラムの流れを紹介する。

1. テーブル全体を撮影し、牛乳パックと容器の位置を認識する。
この撮影は固定した3つの位置です。また、その位置はテーブル全体を出来る限り撮影できるように選択される。
2. 牛乳パック全体の点群を取得する。
3. 牛乳パックの方向を検出する。
4. 左アームが牛乳パックの方向を修正し、テーブルの中央へと移動させる。
5. 右アームが牛乳パックを掴み、左アームの動作に支障のない位置へと移動させる。
6. 容器を上から撮影し、容器の方向と注ぎ位置 P_{pour} を検出する。
7. 容器の方向に合わせて、容器の内部の点群 P_{cup} を取得する。
8. 液面高さから体積への変換テーブル T を作成する。
9. 容器の AABB(Axis Align Bounding Box) を計算する。
10. 左アームを注ぎ動作に干渉せずに容器の中身が見れる位置に移動する。
11. 右アームを注ぎ位置に移動し、左アームのカメラで液面を検出しながら注ぐ。

5.3.2 注ぐ精度の評価実験 1

本実験では、目標容器をテーブルにまっすぐに置いたとき、指定した量を注げるか、その精度を評価する。

実験方法

3種類の容器に対して、それぞれ3通りの目標体積を設定し、注ぐ実験を行った。各容器・目標体積に対して、3回ずつの注ぎ動作を行った。容器のモデルは最初に一度だけ計測し、常に同じモデルを用いた。実験で用いた液体はミルクティーである。各動作ごとに秤で注がれた液体の重量を量り、ミルクティーの密度で割って体積へ変換した。目標体積は容器の体積の $0/3$ – $1/3$ 、 $1/3$ – $2/3$ 、 $2/3$ – $3/3$ の範囲でランダムに選択した。

実験結果

実験の結果は、3回注いだ結果の標準偏差と平均誤差（結果の平均から目標値を引いたもの）で評価する。実験結果は表 5.2 のとおりとなった。

考察

なし。

5.3.3 注ぐ精度の評価実験 2

本実験では、目標容器をテーブルに斜めに置いたとき、指定した量を注げるか、その精度を評価する。

表 5.2: 実験 1 の結果

目標値	注いだ体積			分析	
	1	2	3	標準偏差	平均誤差
容器 1					
64 ml	65.2	65.7	66.2	0.50	1.70
178 ml	183.0	181.9	182.1	0.59	4.33
248 ml	252.0	252.5	253.0	0.50	4.50
容器 2					
27 ml	26.2	26.9	26.5	0.35	-0.47
71 ml	67.8	67.1	68.6	0.75	-3.17
111 ml	104.4	104.6	103.7	0.47	-6.77
容器 3					
12 ml	10.0	8.9	9.0	0.61	-2.70
41 ml	37.0	37.0	36.6	0.23	-4.13
71 ml	66.2	66.3	66.6	0.21	-4.63

実験方法

実験方法は、容器の置き方以外は実験 1 と同様である。ただし、容器には、底にテープでネジを固定し、テーブルに対して 10 度傾けて設置した。また、本実験では 2/3-3/3 の範囲の目標値はあふれる可能性があるため、実験を行わなかった。

実験結果

実験 2 の結果は、表 5.3 のとおりとなった。

表 5.3: 実験 2 の結果

目標値	注いだ体積			分析	
	1	2	3	標準偏差	平均誤差
容器 1					
64 ml	63.7	63.7	64.7	0.58	0.03
178 ml	180.5	179.0	179.0	0.87	1.50
容器 2					
27 ml	22.1	21.8	22.3	0.25	-4.93
71 ml	63.6	64.7	63.5	0.67	-7.07
容器 3					
12 ml	8.9	8.4	9.2	0.40	-3.17
41 ml	36.9	37.0	37.1	0.10	-4.00

考察

5.3.4 考察

それぞれの実験の標準偏差の平均を見ると、立てて置いた際は 0.47 ml、斜めに置いた時は 0.48 ml であり、注ぐ動作の精度が高いことが分かった。しかし、平均誤差の標準偏差は、立てた場合は 4.02 ml、斜めにした場合は 3.19 ml であり、体積測定精度実験の結果をふまえると、注ぐシステムの誤差は主に体積の測定とモデルの精度によるものであると言える。

それぞれの実験で、容器 2 でに対して多くの液体を注ぐときの誤差が -7 ml 程度と大きくなっている。この原因は、注いだ液体の量が容器 2 の容量に近くなると、容器の縁と液面がつながり、一つの平面だと認識され、その結果、液面の高さが真値より高くなったため、注ぐ作業がより早く終わったことと考えられる。

また、注ぐ動作最後の微調整段階では、注ぐ量が少なくなり、液体が目標容器に入らずに元の容器に沿ってこぼれてしまうことがある。

5.4 手法2の実験

5.4.1 角度と出る量の関係の予測精度の実験1

本実験では、角度と出る量の関係を予測するプログラムを抽出し、セクション2.4.2と2.4.1で提案したアルゴリズムの精度を評価する。

実験方法

一回の実験では、予め決めた体積の液体を注ぐ容器に入れ、0から注ぎ終わるまで5度の間隔で回転しつつ、各角度とその角度で出る液体の体積を記録し、その結果を推測した角度と出る量の関係と比較する。

具体的には：

1. 予め決めた体積の水を注ぐ容器に入れる。
2. ロボットアームが注ぐ容器を掴み、注ぐ先容器の上に移動する。
3. 今の姿勢から5度を加え、回転する。
4. 注いだ水の体積を秤で測る。
5. 上の結果を記録する。
6. 液体が注ぎ終わるまで3から5まで繰り返す。

そして、初期液体の体積は、容器容積の $[1/4][2/4]$ や $[3/4]$ とする。

また、注ぐ容器のモデルは予め撮られ、処理した

実験結果

実験結果はしたの図で示した（容器三の実験結果）。横軸は回転角度 [deg]，縦軸は体積 [ml] である。黄色の線はアルゴリズムが予測した結果。緑の線は実際に入れた体積。青い線は予測と実際の差。誤差に集中する図：

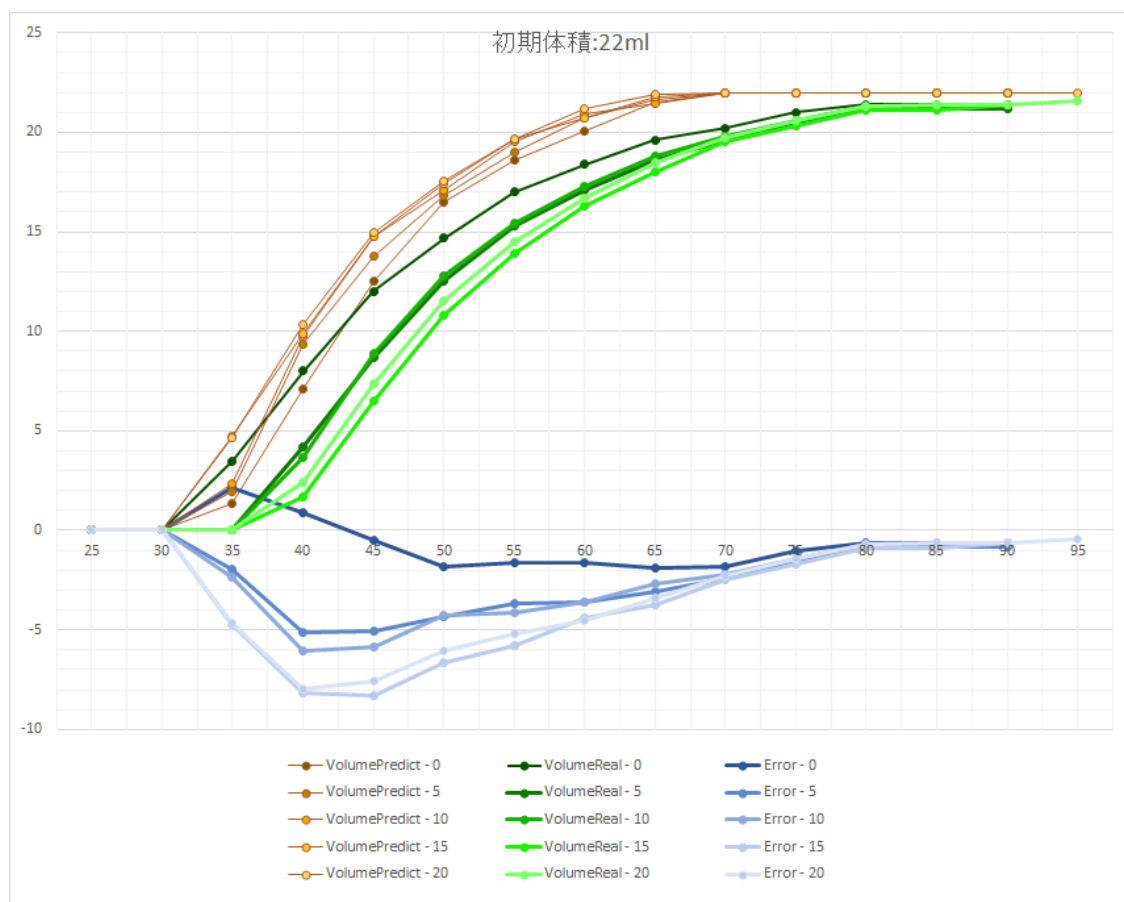


図 5.4: 予測精度の実験 1 の結果 1

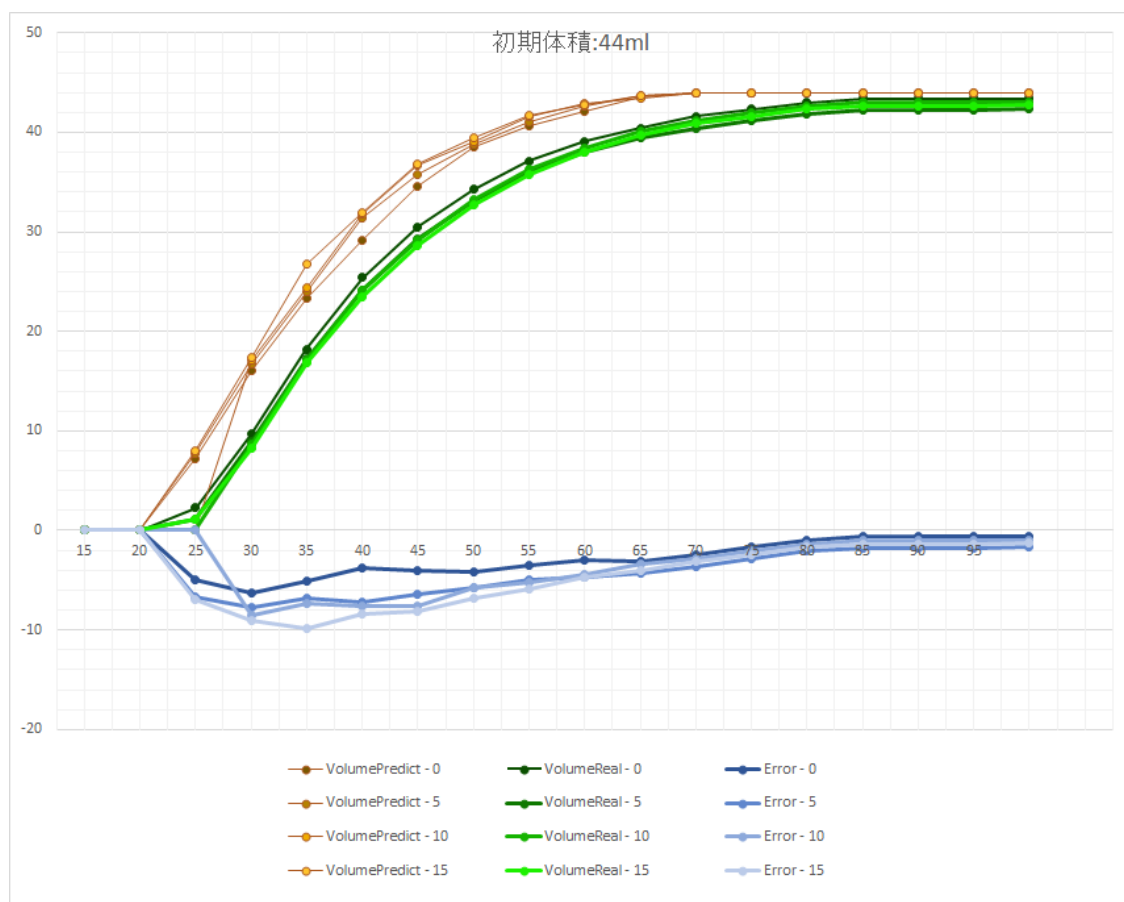


図 5.5: 予測精度の実験 1 の結果 2

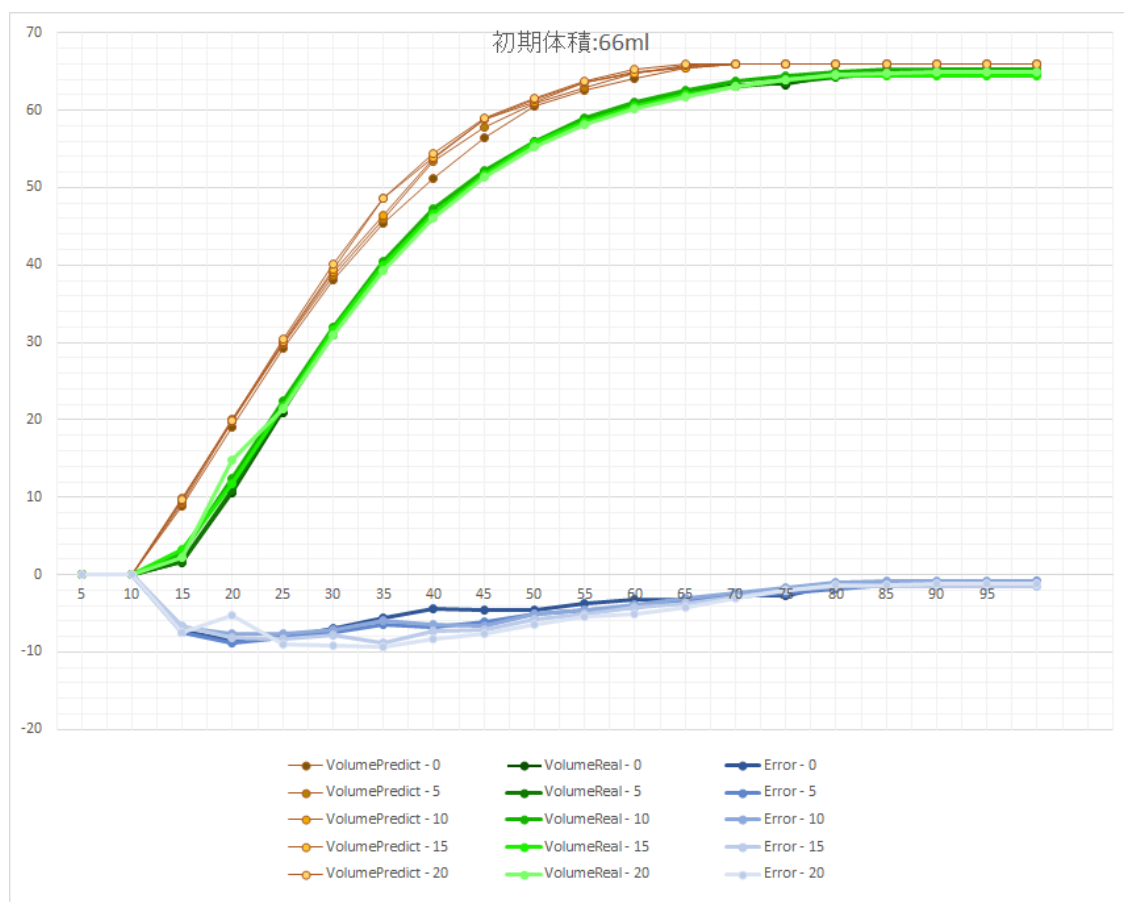


図 5.6: 予測精度の実験 1 の結果 3

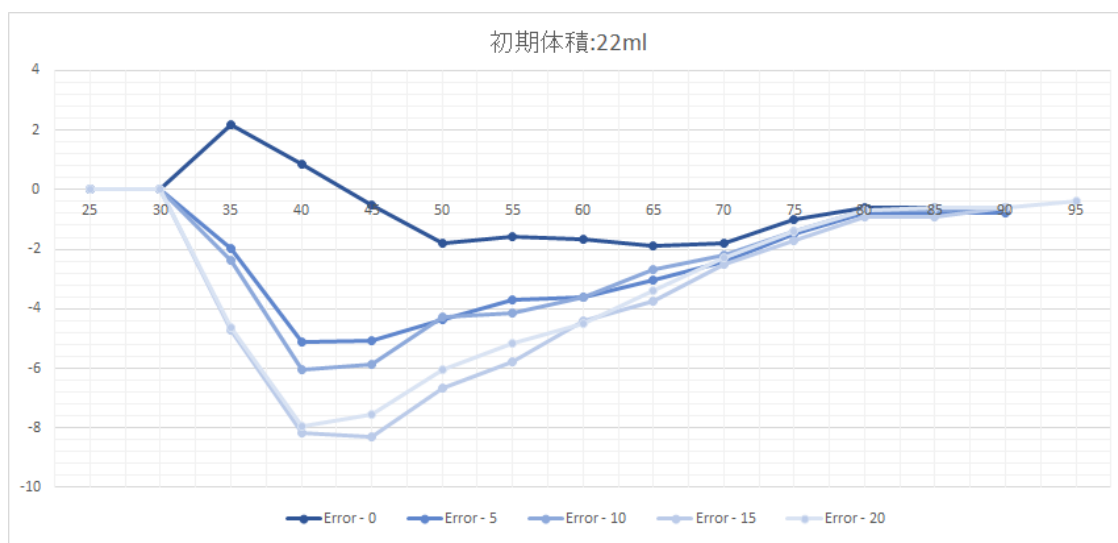


図 5.7: 予測精度の実験 1 の誤差 1

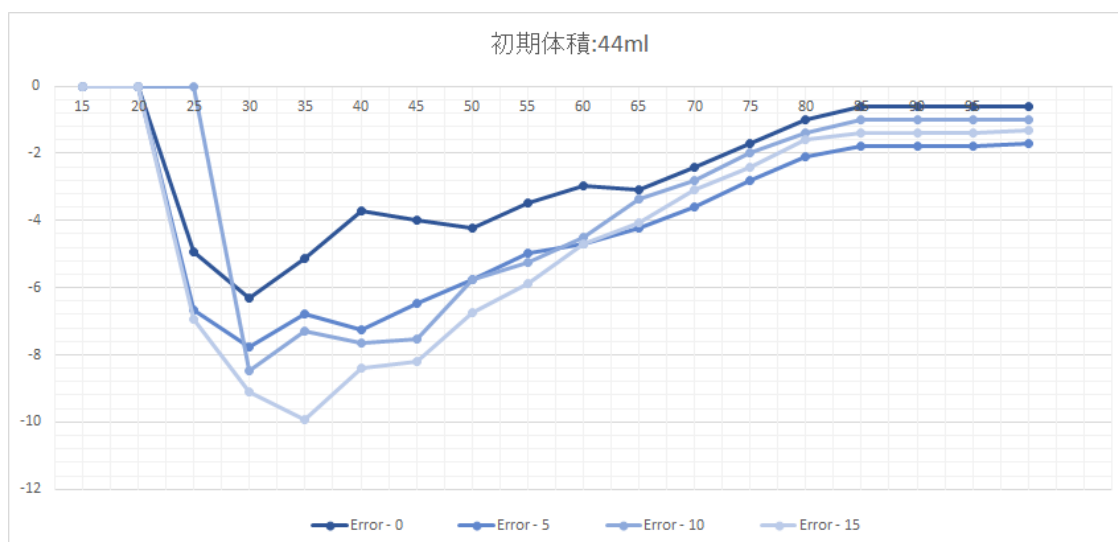


図 5.8: 予測精度の実験 1 の誤差 2

考察

結果の図から見ると、初期体積にもかかわらず、予測の誤差は注ぎ始めるところにほぼ -10[ml] から-5[ml] に集中し、注ぎ作業とともに段々減らす。その誤差の原因に対して、下の図のように、水と容器の表面張力の影響と考えられる。

上の図により、注ぐ口の付近において、液面はほぼ 3[mm] 高くなってしまい、出た量は予測より小さくなる。そして、その誤差は段々減らすことについて、注いでいるうちに、液体

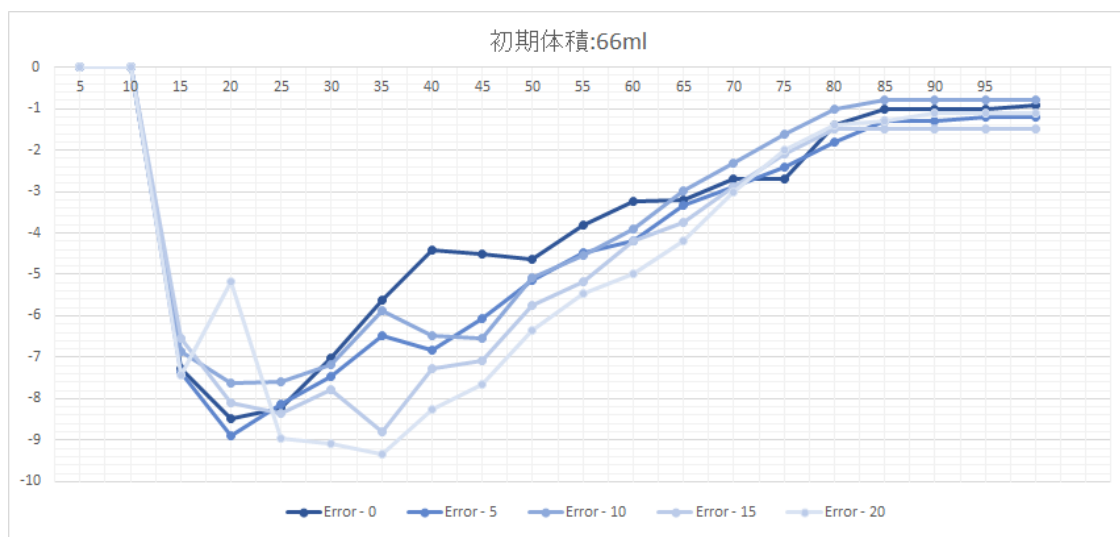


図 5.9: 予測精度の実験 1 の誤差 3



図 5.10: 表面張力

と容器のエッジの接触面積が段々減らしているため、表面張力も減らしている。表面張力が弱くすると、液面は検出した注ぐポイントと近くなって、誤差も小さくなる。

5.4.2 角度と出る量の関係の予測精度の実験 2

セクション 5.4.1 の考察により、液体の流動は表面張力のあるため、非常に困難になる。そこで、アルゴリズムの精度を正しく評価するため、洗剤をコップのエッジにかけ、予測精度実験 2 を実施した。

実験方法

実験方法はセクション 5.4.1 の方法や手順と同じである。ただし、初期体積は $[3/4]$ に固定した。

実験結果

実験結果はしたの図で示した（容器三の実験結果）。

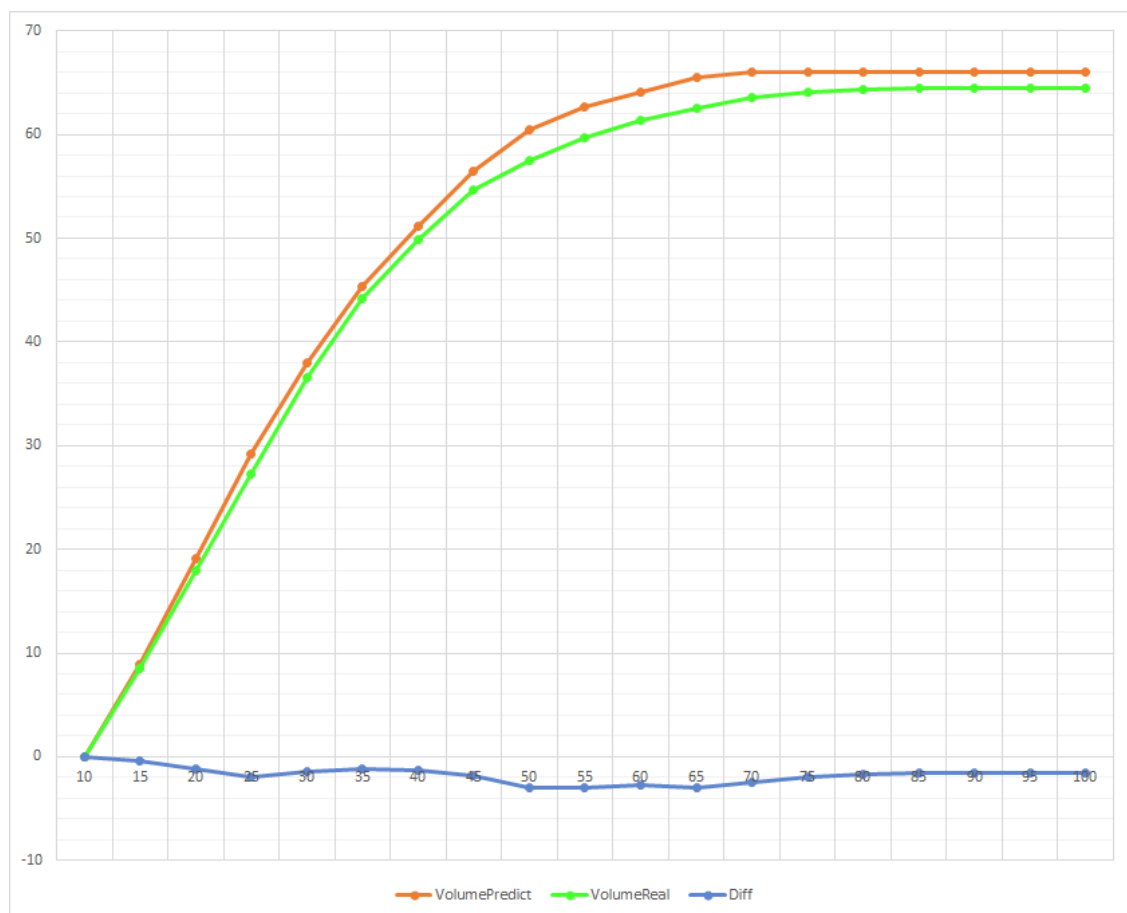


図 5.11: 予測精度の実験 2 の結果

考察

洗剤を使用した結果は下の図のように：

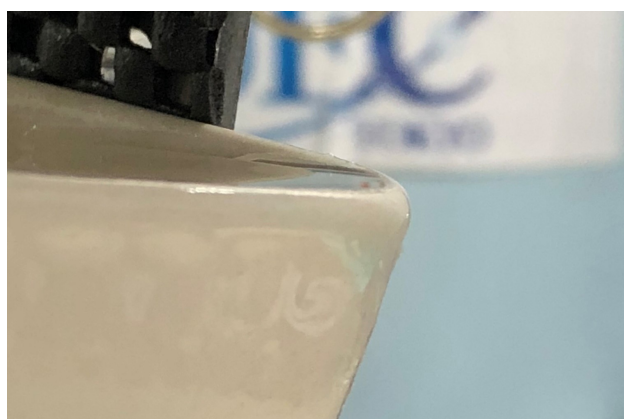


図 5.12: 表面張力をなくした

実験 1 より，表面張力の影響はほぼなくした．

実験の結果について，表面張力の影響がないと，誤差の平均は 1.57，標準偏差は 0.93 である．実験 1 より，誤差が大きく減少した．

5.4.3 注ぐ精度の評価実験 3 TODO:実験 3 未完成

注ぐ容器モデルを取得することやその中に入った液体の量を測定することは全部プログラムに書き込み，セクション 2.4 で提案した手法 2 が指定した量を注げるか，その全体的な精度を評価する．

実験方法

注ぐ容器モデルを取得することやその中に入った液体の量を測定することは全部プログラムに書き込む．

実験結果

考察

5.5 考察

なになに

第6章 結言

6.1 まとめ

本稿では，ロボットにより容器に液体を定量的に注ぐための手法として，RGB-D カメラを使用して目標容器の形状を取得し，それを用いて注がれた液体の体積を推定することによって指定した量の液体を注ぐ手法を提案した．そして，その手法を双腕ロボットに実装し，体積測定実験と注ぐ実験が行い，計3種類の目標容器に定量の液体を注いだ．実験から得られたデータで，平均 ± 4 ml の誤差があるが，日常生活において十分な精度であると考えられる．したがって，本稿で提案した手法は容器に液体を定量的に注ぐことができる．

本論文では，双腕ロボットと視覚情報を用いて不透明な液体を目標容器に定量的に注ぐ手法を提案し，実装した．そして，5つの実験により，手法1（先行研究より5倍以上の精度）と手法2は定量的に注げることが確認した．したがって，本研究は注ぐ容器と注ぎ先の容器のモデルどちらかが計測できる条件で不透明な液体を目標容器に定量的に注ぐことが実現した．

6.2 今後の展望

ConvexHull ではなく，ConcaveHull を使い，より複雑な形状の容器にも対応できる．

謝辞

本研究を進めるにあたり，様々な視点から多くのご助言やご指導を頂いた末廣尚士教授，工藤俊亮准教授に深く感謝の意を表します．また，研究班の先輩としてご指導をいただきました長野恵典さん，何成浩さんに感謝の意を表します．

最後になりますが，研究や日々の学生生活など様々な場面でお世話になった知能システム学講座の皆さまのおかげで充実した研究室生活を送ることができました．ありがとうございました